MAR ATHANASIUS COLLEGE OF ENGINEERING

KOTHAMANGALAM

(Government Aided Autonomous Institution)

Department of Computer Applications

Mini Project Report

SUPPLYGUARD: AN AI-POWERED SHIPMENT DELAY PREDICTION AND RISK ANALYSIS SYSTEM

Done by

ADON RENJI

Reg No: MAC25MCA-2006

Under the guidance of

Prof. Sibu Skaria

2025-2027

MAR ATHANASIUS COLLEGE OF ENGINEERING
KOTHAMANGALAM

(Government Aided Autonomous Institution)

CERTIFICATE

SUPPLYGUARD: AN AI-POWERED SHIPMENT DELAY PREDICTION AND RISK ANALYSIS SYSTEM

Certified that this is the bonafide record of project work done by

ADON RENJI

Reg No: MAC25MCA-2006

during the third semester, in partial fulfilment of requirements for award of the degree,

Master of Computer Applications

of

APJ Abdul Kalam Technological University, Thiruvananthapuram

Prof. Sibu Skaria
Faculty Guide

Dr. Sonia Abraham
Head of the Department

Dr. Manu John
Project Coordinator

Internal Examiners

Acknowledgement

With heartfelt gratitude, I extend my deepest thanks to the Almighty for His unwavering grace and blessings that have made this journey possible. May His guidance continue to illuminate my path in the years ahead.

I am immensely thankful to Dr. Sonia Abraham, Head of the Department of Computer Applications, Prof. Sibu Skaria, my mini project guide, Dr. Manu John and Prof. Minu Maria Joseph, our dedicated project coordinators, for their invaluable guidance and timely advice, which played a pivotal role in shaping this project. Their guidance, constant supervision, and provision of essential information were instrumental in the successful completion of the mini project.

I extend my profound thanks to all the professors in the department and the entire staff at MACE for their unwavering support and inspiration throughout my academic journey. My sincere appreciation goes to my beloved parents, whose guidance has been a beacon in every step of my path.

I am also grateful to my friends and individuals who generously shared their expertise and assistance, contributing significantly to the fulfillment of this endeavor.

Adon Renji

Abstract

Supply chain logistics and freight transportation face severe operational disruptions and financial penalties caused by unforeseen delivery delays. Traditional supply chain monitoring systems rely primarily on retrospective reporting and manual dispatch assessments, which fail to detect bottlenecks before shipments leave the warehouse facility. Machine Learning offers a transformative methodology to forecast late delivery risk proactively by learning complex logistical patterns from historical order data.

This project develops and evaluates SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System, an enterprise predictive analytics platform built on the global DataCo Smart Supply Chain Dataset (180,519 records). A fundamental design principle of SupplyGuard is the complete isolation of 19 strictly pre-dispatch features available at order placement—including shipping mode, scheduled transit days, destination geography, item pricing, order timing, and temporal interaction variables—thereby completely eliminating post-dispatch target leakage. Two machine learning classifiers, Random Forest and Support Vector Machine (SVM), were developed and benchmarked on an 80:20 stratified holdout partition (36,104 unseen test shipments). The optimized Random Forest ensemble (200 estimators) substantially outperformed the SVM baseline, achieving an overall classification accuracy of 83.89% and an ROC-AUC of 91.18%. Calibrating the classification decision threshold to 0.41 yielded a high delay recall of 88.31% on the test partition ensuring that critical supply chain delays are captured before parcels enter transit.

The trained machine learning model is deployed as a web application using Python Flask . The system uses role-based access control (RBAC) to provide separate interfaces for customers and managers. Customers can place orders through a simple three-step checkout process, while managers can monitor shipment risks through a dashboard. The dashboard displays late delivery probabilities, visual risk indicators, key causes of potential delays, and suggested actions to reduce delivery risks.

List of Tables

Table 2.1: Summary of Paper 1 4
Table 2.2: Summary of Paper 2 6
Table 2.3: Summary of Paper 3 8
Table 2.4: Overall Summary of Selected Research Papers 9
Table 4.1: Model Hyperparameter Configuration 40
Table 5.1: Performance Comparison of Random Forest and SVM on the Test Set 45
Table 5.2: Confusion Matrix Values of Random Forest 49
Table 5.3: Per-Class Metrics Calculated from the Confusion Matrix 51
Table 5.4: Misclassifications by Type 52
Table 5.5: Prediction Statistics of the Selected Random Forest Model on Unseen Test Data . 53
Table 10.1: Minimum Software Requirements 64
Table 10.2: Minimum Hardware Requirements 64

List of Figures

Figure 3.1: Preview of the DataCo Smart Supply Chain Dataset 15
Figure 3.2: Information of the Dataset 17
Figure 3.3: Shape of the Dataset 17
Figure 3.4: Dataset Attributes (Column Names) 17
Figure 3.5: Statistical Summary 18
Figure 3.6: Missing Values in Dataset Attributes 19
Figure 3.7: Checking Duplicate Records 19
Figure 3.8: Outliers Identified in Dataset 20
Figure 3.9: One-Hot Encoding Transformation of Categorical Features 20
Figure 3.10: Feature Variables Selected for Model Development 21
Figure 3.11: Summary of Feature Variables 23
Figure 3.12: Summary of Class Variable Distribution 24
Figure 3.13: Data Visualization of Key Features 24
Figure 3.14: Activity Diagram of the Proposed SupplyGuard System 29
Figure 3.15: Project Pipeline of the Proposed SupplyGuard System 32
Figure 5.1: Graphical Comparison of Random Forest and SVM 46
Figure 5.2: Confusion Matrix of Random Forest 49
Figure 5.3: Confusion Matrix of SVM 50
Figure 6.1: Authentication and Secure Role-Based Login Screen (/login) 56
Figure 6.2: Manager Operations Dashboard and Order Lookup Cockpit (/) 56
Figure 6.3: Late Delivery Risk Assessment Report with Circular Risk Gauge 57
Figure 6.4: Machine Learning Prediction Explanation UI 58
Figure 7.1: Overall Project Repository on GitHub 59
Figure 7.2: Directory Structure of the Project on GitHub 60
Figure 7.3: Source Code of the Application on GitHub 61

Contents

Acknowledgement i

Abstract ii

List of Tables iii

List of Figures iv

1. Introduction 1

2. Supporting Literature 2

2.1 Literature Review 2

2.1.1 Paper 1 - A Novel Machine Learning Model for Late Supplier Deliveries 2

2.1.2 Paper 2 - Integrating Machine Learning and Deep Learning for Supplier Risk ... 4

2.1.3 Paper 3 - A Deep Learning Approach to Predict Delivery Delays 6

2.2 Literature Review Summary 8

2.3 Findings and Proposals 9

2.4 Problem Statement 10

2.5 Research Gap 11

2.6 Motivation 12

3. System Analysis 13

3.1 Analysis of Dataset 13

3.1.1 About the Dataset 13

3.1.2 Explore the Dataset 15

3.2 Data Preprocessing 18

3.2.1 Data Cleaning 18

3.2.2 Analysis of Feature Variables 21

3.2.3 Analysis of Class Variables 23

3.3 Data Visualization 24

3.4 Analysis of Architecture and Algorithms 24

3.4.1 Detailed Study Report of Proposed Algorithms 25

3.4.2 Activity Diagram to Represent the Flow of Algorithm(s) 28

3.4.3 Step-by-Step Explanation about Working of Algorithm(s) 30

3.5 Project Pipeline 32
3.6 Feasibility Analysis 34
3.6.1 Technical Feasibility 34
3.6.2 Economic Feasibility 35
3.6.3 Operational Feasibility 35
3.7 System Environment 35
3.7.1 Software Environment 36
3.7.2 Hardware Environment 36
4. System Design **38**
4.1 Model Building 38
4.1.1 Implementation Code 38
4.1.2 Model Planning (Training/Testing Split of Dataset) 41
4.1.3 Training - Build Model Using Train Dataset 42
4.1.4 Testing - Evaluate Performance Using Test Dataset 43
5. Results and Discussion **45**
5.1 Comparison of Models 45
5.2 Evaluation Metrics 46
5.3 Confusion Matrix 48
5.3.1 Random Forest Confusion Matrix 48
5.3.2 SVM Confusion Matrix 49
5.4 Relevant Metrics Calculated from Confusion Matrix 50
5.4.1 Misclassification Analysis 52
5.5 Prediction Statistics of Unseen Data 52
5.6 Discussion of Findings and Conclusions 53
6. Model Deployment **55**
6.1 Model Implementation & System Architecture 55
6.2 User Interface Design 55
7. Git History **59**
7.1 Overall Project Repository 59
7.2 Project Directory Structure 60
7.3 Source Code Repository 60

8. Conclusions **62**

9. Future Scope **63**

10. Appendix **64**

10.1 Minimum Software Requirements 64

10.2 Minimum Hardware Requirements 64

11. References **65**

1. Introduction

The rapid growth of global e-commerce has increased the complexity of supply chain and shipment management. Timely delivery is important for maintaining customer satisfaction and operational efficiency. However, shipment delays can occur due to various factors related to shipping methods, customer information, product characteristics, geographical conditions, and order timing. Predicting the possibility of delay in advance can help improve supply chain decision-making.

Traditional approaches often depend on manual monitoring and historical analysis to identify delivery problems. When large volumes of supply chain data are involved, these approaches can become time-consuming. Machine Learning provides an effective approach by learning patterns from historical shipment records and using them to predict future delivery risks.

SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System is a Machine Learning-based project developed to predict shipment late-delivery risk using historical supply chain data. The project uses the DataCo Smart Supply Chain Dataset, containing 180,519 records and 53 attributes related to customers, products, orders, shipments, markets, and geographical information.

The target variable used is Late_delivery_risk, which indicates whether a shipment is associated with late-delivery risk. Relevant shipment, customer, product, market, geographical, and temporal features are selected for model development. Calendar-based features are also derived from order-date information, along with an interaction feature between shipping mode and order hour.

Two Machine Learning algorithms, Random Forest and Support Vector Machine (SVM), are evaluated for late-delivery-risk classification. Their performance is compared using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. The final system uses a Random Forest model with 200 decision trees with a tuned decision threshold of 0.41.

A web-based interface is provided to demonstrate the trained model. Users can enter the required shipment information through the webpage, and the saved Machine Learning pipeline generates a late-delivery probability and corresponding risk prediction.

2. Supporting Literature

2.1 Literature Review

Paper 1 - A Novel Machine Learning Model for Predicting Late Supplier Deliveries of Low-Volume-High-Variety Products with Application in a German Machinery Industry

The research titled “A Novel Machine Learning Model for Predicting Late Supplier Deliveries of Low-Volume-High-Variety Products with Application in a German Machinery Industry” focuses on predicting supplier delivery delays in a low-volume-high-variety manufacturing environment. The study highlights the importance of using Machine Learning techniques to analyze purchasing data and identify patterns associated with late supplier deliveries. Early prediction of potential delivery delays can help manufacturing organizations improve procurement and production planning.

The proposed approach uses real purchasing data collected from a German machinery manufacturer. The study builds a detailed understanding of the manufacturer's procurement process and identifies relevant features that can be used for predicting supplier delivery performance. The data is analyzed and prepared before applying different Machine Learning algorithms to determine their suitability for predicting the severity of late deliveries.

The researchers evaluate several supervised Machine Learning algorithms and develop a regression-based prediction model for estimating the severity of supplier delivery delays. The study demonstrates that prediction is possible during earlier phases of the purchasing process. It also shows that high-dimensional input features do not necessarily require dimensionality reduction, allowing a larger set of relevant information to be considered during prediction.

The study shows that Machine Learning-based prediction can help organizations identify potential delivery problems at an early stage and support better procurement and production planning. However, the research mainly focuses on predicting the severity of supplier delivery delays and does not provide pre-dispatch risk factor analysis or an integrated manager approval workflow. The proposed SupplyGuard system extends this concept by combining shipment delay-risk prediction with an automated risk analysis module evaluating shipping parameters and transit buffers, providing a practical decision-support solution for logistics management.

TITLE	A Novel Machine Learning Model for Predicting Late Supplier Deliveries of Low-Volume-High-Variety Products with Application in a German Machinery Industry
AREA OF WORK	Development of a Machine Learning-based regression framework for predicting the severity of late supplier deliveries in a low-volume-high-variety manufacturing environment to support procurement and production planning.
DATASET	Real purchasing and supplier delivery data collected from a German machinery manufacturer, representing a low-volume-high-variety production environment and containing information related to the purchasing process and supplier deliveries.
METHODOLOGY / STRATEGY	Data analysis, procurement-process understanding, relevant feature identification, preparation of purchasing data, supervised Machine Learning model development, comparison of different Machine Learning algorithms, and evaluation of their ability to predict the severity of supplier delivery delays.
ALGORITHM	Regression-based Machine Learning models • Multiple Machine Learning algorithms were compared to identify a suitable prediction approach.
RESULT / PRECISION	The study demonstrates that Machine Learning can predict the severity of supplier delivery delays during earlier phases of the purchasing process. It also shows that high-dimensional input features do not necessarily require dimensionality reduction for the proposed prediction approach.
ADVANTAGES	The proposed approach enables organizations to predict potential supplier delivery delays at an early stage, supporting better procurement and production planning. It makes use of high-dimensional purchasing information without requiring mandatory dimensionality reduction and can help organizations take preventive actions to reduce the impact of supplier delays.

FUTURE PROPOSAL	Future improvements can include the integration of additional information such as component identifiers and other relevant procurement-related features to improve prediction quality and make the model more effective for practical supplier-delivery prediction.

Table 2.1: Summary of Paper 1

Paper 2 - Integrating Machine Learning and Deep Learning for Enhanced Supplier Risk Prediction

The research titled "Integrating Machine Learning and Deep Learning for Enhanced Supplier Risk Prediction in Supply Chain Management" focuses on improving supplier selection and risk assessment using Artificial Intelligence techniques. Supplier-related issues such as delayed deliveries, inconsistent product quality, and capacity constraints can significantly affect the overall performance of a supply chain. The study highlights the importance of identifying high-risk suppliers at an early stage to ensure uninterrupted logistics operations and maintain customer satisfaction. By integrating Machine Learning and Deep Learning models, the researchers aim to develop a robust framework for predicting supplier risks and supporting strategic procurement decisions.

The proposed approach uses historical supplier data collected from an automotive manufacturer's ERP system. The dataset contains information related to suppliers, quality issues, failure descriptions, issue severity, plant location, replacement time, recurrent issues, total cost, response time, and other supplier-related factors. The data is processed and relevant features are selected before applying the Machine Learning and Deep Learning models.

The researchers evaluate multiple models for both classification and regression tasks. The classification models include Random Forest, SVM, KNN, and ANN, while regression models include Random Forest Regressor, SVR, Linear Regression, and ANN. The models are compared using appropriate evaluation metrics to determine their effectiveness in predicting supplier failures, number of issues, and disruption costs.

The study shows that Machine Learning can effectively identify supply chain risks and support proactive operations. However, the research mainly focuses on supplier failure prediction and does not provide an integrated shipment-delay prediction and risk analysis system. The proposed SupplyGuard system extends this concept by combining shipment delay-risk

prediction with automated root-cause risk analysis based on shipping modes, schedule buffers, and regional destinations, providing a practical decision-support solution for logistics management.

TITLE	Integrating Machine Learning and Deep Learning for Enhanced Supplier Risk Prediction
AREA OF WORK	Application of Machine Learning and Deep Learning techniques for predicting supplier-related risks, failures, issues, and disruption costs in supply chain management.
DATASET	The study utilizes historical supplier performance data containing supplier reliability, delivery history, production capacity, order completion rates, quality metrics, and customer feedback.
METHODOLOGY / STRATEGY	The dataset is preprocessed through data cleaning, normalization, feature engineering, and missing value handling. Machine Learning and Deep Learning models are trained and evaluated using classification performance metrics to identify supplier risks accurately.
ALGORITHM	Random Forest, SVM, KNN, ANN, Random Forest Regressor, SVR, Linear Regression
RESULT / PRECISION	The integrated Machine Learning and Deep Learning framework achieves high prediction accuracy by effectively classifying suppliers based on their reliability and operational risk, thereby improving procurement decisions.
ADVANTAGES	The proposed approach enables organizations to identify high-risk suppliers before procurement decisions are made. It improves supplier evaluation, reduces procurement risks, enhances supply chain resilience, and supports better decision-making by utilizing historical supplier performance data.
FUTURE PROPOSAL	Future enhancements include integrating real-time supplier performance monitoring, IoT-based inventory tracking, external market indicators, and explainable Artificial Intelligence

	techniques to improve prediction transparency and decision-making accuracy.

Table 2.2: Summary of Paper 2

Paper 3 - A Deep Learning Approach to Predict Supply Chain Delivery Delays Based on Macroeconomic Indicators

The research titled "A Deep Learning Approach to Predict Supply Chain Delivery Delays Based on Macroeconomic Indicators" investigates the use of Deep Learning techniques to improve the prediction of shipment delivery delays in complex supply chain environments. The study emphasizes that conventional prediction models primarily rely on historical shipment records and often overlook external factors that influence delivery performance. To address this limitation, the researchers propose a predictive framework that combines logistics data with macroeconomic indicators such as fuel prices, inflation rates, transportation costs, and economic conditions to achieve more accurate delay predictions.

The proposed system collects historical shipment information along with relevant economic indicators that may affect transportation and logistics operations. Before model training, the collected data undergoes preprocessing steps including data cleaning, feature selection, normalization, and transformation of categorical variables into numerical representations. The Deep Learning model is then trained to identify hidden relationships between shipment characteristics and external economic factors that contribute to delivery delays. Performance is evaluated using standard metrics such as Accuracy, Precision, Recall, F1-Score, and Mean Absolute Error (MAE).

Experimental results indicate that Deep Learning models outperform several traditional Machine Learning algorithms by effectively capturing complex nonlinear relationships within the dataset. The study demonstrates that incorporating macroeconomic indicators significantly improves prediction accuracy and enables organizations to anticipate delivery disruptions more effectively. However, the proposed framework mainly focuses on macro-level delay forecasting and does not include order-level risk factor analysis or actionable dispatch mitigation guidance. The proposed SupplyGuard system builds upon this concept by integrating shipment delay prediction with granular risk analysis and operational mitigation recommendations, thereby providing a more comprehensive logistics decision-support solution.

TITLE	A Deep Learning Approach to Predict Supply Chain Delivery Delays Based on Macroeconomic Indicators
AREA OF WORK	The research focuses on predicting supply chain delivery delays using Deep Learning techniques by combining historical delivery information with macroeconomic indicators that may influence supply chain and transportation performance.
DATASET	The study utilizes historical shipment records along with macroeconomic data such as fuel prices, transportation costs, inflation rates, economic conditions, and delivery schedules.
METHODOLOGY / STRATEGY	The collected data is preprocessed through cleaning, normalization, feature engineering, and encoding of categorical variables. A Deep Learning model is trained to identify hidden relationships between shipment attributes and external economic factors affecting delivery performance.
ALGORITHM	Deep Neural Network (DNN), Long Short-Term Memory (LSTM), and Artificial Neural Network (ANN) models are implemented to predict shipment delivery delays.
RESULT / PRECISION	The Deep Learning models achieve higher prediction accuracy by incorporating macroeconomic indicators, enabling organizations to identify potential shipment delays more effectively than traditional prediction methods.
ADVANTAGES	The proposed approach improves the accuracy of shipment delay prediction by considering both historical logistics data and external economic factors. It enables proactive logistics planning, reduces transportation risks, enhances resource utilization, and supports informed decision-making for supply chain managers.
FUTURE PROPOSAL	Future work recommends integrating real-time weather information, GPS tracking, traffic conditions, IoT sensor data, and supplier performance metrics to develop a more intelligent

	and adaptive supply chain prediction system capable of responding to dynamic logistics environments.

Table 2.3: Summary of Paper 3

2.2 Literature Review Summary

TITLE	SUMMARY
Fabian Steinberg et al., "A Novel Machine Learning Model for Predicting Late Supplier Deliveries of Low-Volume-High-Variety Products with Application in a German Machinery Industry," Supply Chain Analytics, 2023	This study focuses on predicting the severity of late supplier deliveries in a low-volume-high-variety manufacturing environment. The authors develop a Machine Learning-based regression model using purchasing data from a German machinery manufacturer and compare different Machine Learning algorithms to identify the most suitable prediction approach. The study demonstrates that supplier delivery delays can be predicted during earlier phases of the purchasing process and that high-dimensional input features do not necessarily require dimensionality reduction. The research shows the potential of predictive analytics for improving procurement and production planning. However, the study focuses mainly on predicting supplier delivery delay severity and does not include pre-dispatch risk factor analysis, operational mitigation guidance, or manager approval within an integrated decision-support system.
N. Rezki and M. Mansouri, "Integrating Machine Learning and Deep Learning for Enhanced Supplier Risk Prediction."	This research focuses on predicting supplier risks using Artificial Intelligence techniques. Historical supplier performance data is analyzed

	using Machine Learning and Deep Learning algorithms to identify reliable suppliers and minimize procurement risks. The study demonstrates that AI-based supplier evaluation improves procurement decisions and strengthens supply chain resilience. However, the framework does not include shipment delay prediction or integrated logistics management.
Gabellini et al., "A Deep Learning Approach to Predict Supply Chain Delivery Delays Based on Macroeconomic Indicators.	This paper proposes a Deep Learning model that combines historical shipment records with macroeconomic indicators such as fuel prices and transportation costs to predict delivery delays. The results show that Deep Learning improves prediction accuracy by capturing complex relationships between logistics and external factors. Nevertheless, the study does not provide order-level risk factor analysis or operational dispatch mitigation guidance, leaving scope for a more comprehensive logistics decision-support system.

Table 2.4: Overall Summary of Selected Research Papers

2.3 Findings and Proposals

The literature review reveals that Artificial Intelligence and Machine Learning have become increasingly important in improving supply chain management. The reviewed studies demonstrate that predictive models can analyze historical logistics data to identify shipment delays and assess supplier-related risks. These approaches help organizations improve operational efficiency, reduce supply chain risks, and support proactive decision-making.

The first research paper highlights the application of several Machine Learning algorithms, including Random Forest, Gradient Boosting, KNN, Decision Tree, Logistic Regression, Linear Regression, and SVM, for proactive supply chain risk management and shipment-delay prediction. The second paper focuses on supplier risk prediction using Machine Learning and Deep Learning techniques, enabling organizations to identify supplier-related risks and support

better procurement decisions. The third paper further explores shipment-delay prediction using an LSTM-based Deep Learning approach combined with macroeconomic indicators, demonstrating the importance of considering external factors in supply chain prediction.

Although these studies contribute significantly to intelligent supply chain management, several limitations are observed. Most existing systems concentrate on retrospective reporting or generic delay forecasting. Few approaches integrate shipment-delay prediction with granular operational risk analysis in a unified decision-support system. In addition, existing approaches do not directly provide actionable dispatch mitigation recommendations while allowing a logistics manager to make informed approval and routing decisions.

Based on these findings, the proposed SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System is designed to address these limitations. The system evaluates Random Forest and Support Vector Machine (SVM) for shipment delay-risk prediction using selected historical logistics features. In addition to delay prediction, the system provides an automated risk analysis module evaluating shipping mode constraints, transit schedules, and regional routing factors. This integrated approach helps logistics managers identify potential delivery risks, understand root causes, and take proactive mitigation countermeasures.

The proposed system is implemented using Python Flask for the backend web server, responsive HTML5/CSS3/JavaScript for the frontend user portals, an SQLite relational database (supplyguard.db) for shipment and user order storage, and Machine Learning models developed in Python using scikit-learn and Joblib. The integration of predictive analytics, automated risk analysis, shipment storage, and manager approval provides a practical decision-support solution for proactive supply chain management.

2.4 Problem Statement

Supply chain management plays a crucial role in ensuring the timely delivery of products and maintaining customer satisfaction. However, shipment delays caused by supplier-related issues, transportation conditions, operational problems, and improper logistics planning continue to be major challenges for organizations. These delays can increase operational costs, disrupt business activities, and negatively affect customer satisfaction.

Most existing logistics management systems are designed to monitor shipment status after dispatch rather than predict potential delays in advance. As a result, logistics managers may

become aware of delivery problems only after they have occurred, leaving limited opportunities to take preventive actions. In addition, supplier selection is often based on manual evaluation or historical experience, which may not sufficiently consider the current reliability and workload of available suppliers.

The absence of an integrated decision-support system that combines shipment delay prediction with comprehensive risk analysis makes it difficult for organizations to proactively manage delivery risks. Therefore, there is a need for a system that can predict shipment delay risk using Machine Learning techniques and provide granular risk factor analysis based on shipping parameters, schedule buffers, and regional routes. Such a system can help managers take proactive measures, improve logistics planning, mitigate delay risks, and enhance the overall efficiency of supply chain operations.

2.5 Research Gap

The literature review indicates that considerable research has been conducted on shipment delay prediction and supplier risk assessment using Machine Learning and Deep Learning techniques. These studies have demonstrated the effectiveness of predictive models in improving logistics planning, reducing operational risks, and enhancing supply chain performance. However, several limitations still exist in the current approaches.

Most existing research focuses mainly on predicting shipment delays without providing actionable operational recommendations to mitigate the risk before dispatch. Similarly, studies related to supply chain risk assessment mainly focus on macro-level disruptions but do not integrate order-specific delay prediction into the decision-making process. As a result, logistics managers may need to use separate systems or manual analysis to make operational decisions.

Another significant gap is the limited integration of automated risk analysis and operational mitigation guidance within existing logistics software. Existing approaches generally focus on post-dispatch tracking and lack an integrated platform that combines predictive analytics with actionable pre-dispatch risk analysis. Furthermore, many approaches do not provide practical decision-support features that can help organizations proactively respond to potential shipment delays.

To address these research gaps, the proposed SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System integrates shipment delay prediction with granular risk factor analysis in a single platform. By utilizing Random Forest and Support Vector Machine

(SVM) algorithms along with feature-level risk attribution, the proposed system provides a comprehensive decision-support solution that enables organizations to reduce delivery delays, identify root-cause vulnerabilities, and enhance the overall efficiency of supply chain management.

2.6 Motivation

The rapid growth of global trade and e-commerce has increased the demand for efficient and reliable supply chain management systems. Organizations are required to ensure timely delivery of products while minimizing operational costs and maintaining high levels of customer satisfaction. However, shipment delays caused by shipping mode bottlenecks, scheduling constraints, regional transit hubs, and ineffective logistics planning continue to pose significant challenges to businesses.

Existing logistics systems primarily focus on tracking shipments after dispatch rather than predicting potential delays before they occur. Moreover, dispatch risk evaluation is often based on manual estimation or retrospective reporting, which fails to capture complex non-linear interactions across shipping parameters. These limitations reduce the ability of organizations to make proactive decisions and respond effectively to potential supply chain disruptions.

The motivation behind developing the SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System is to provide an intelligent decision-support platform that enables organizations to anticipate shipment delays and analyze operational risk factors before delivery issues arise. By leveraging Machine Learning algorithms such as Random Forest and Support Vector Machine (SVM), the system analyzes historical shipment data to predict delay risks with high accuracy.

In addition to delay prediction, the proposed system evaluates key risk drivers—including shipping modes, scheduled delivery buffers, and destination regions—to provide targeted operational mitigation recommendations for each shipment. This integrated approach helps organizations improve logistics planning, optimize dispatch scheduling, reduce operational costs, minimize delivery failures, and enhance customer satisfaction. The project demonstrates how Artificial Intelligence can be effectively applied to modern supply chain management to achieve smarter, faster, and more reliable logistics operations.

3. System Analysis

System analysis is the process of examining the existing system, identifying its limitations, and designing an efficient solution to meet the required objectives. It helps in understanding the system requirements, data flow, and overall functionality before implementation.

The proposed SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System analyzes historical shipment data to predict delivery delays and evaluate key operational risk factors. The system utilizes Machine Learning techniques to improve logistics planning, reduce operational risks, and support better decision-making. This chapter discusses the dataset, data preprocessing, visualization, system architecture, algorithms, project pipeline, and feasibility analysis of the proposed system.

3.1 Analysis of Dataset

3.1.1 About the Dataset

a) Dataset Overview

The SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System utilizes the DataCo Smart Supply Chain Dataset to develop Machine Learning models for predicting shipment delays and supporting comprehensive risk analysis. The dataset contains historical supply chain records collected from logistics operations and is widely used for research in supply chain analytics and predictive modeling. It includes information related to customer orders, products, suppliers, shipments, transportation, delivery status, and sales transactions.

The dataset consists of 180,519 records and 53 attributes, containing both numerical and categorical features. The numerical attributes include sales amount, shipping cost, order item quantity, scheduled delivery days, actual delivery days, benefit per order, and product price. The categorical attributes include shipping mode, order status, delivery status, customer segment, product category, supplier information, order region, and market. These attributes are used to identify patterns that influence shipment delays and logistics performance.

The target variable used in the proposed system is Late_delivery_risk, which is a binary classification attribute indicating whether a shipment is delayed (1) or delivered on time (0). The dataset is well suited for supervised Machine Learning classification and provides

sufficient information to analyze the factors affecting shipment delays and supplier performance.

Name	DataCo Smart Supply Chain Dataset
Source	Kaggle
Size	180,519 Records
Attributes	53
Predictor Variables	19 pre-dispatch variables
Target Variable	Late_delivery_risk
Target Classes	0 (No Late Risk), 1 (Late Risk)
Class Distribution	Class 0: 81,542 (45.17%), Class 1: 98,977 (54.83%)

The final Machine Learning pipeline utilizes 19 pre-dispatch features available at order placement, organized into the following operational categories:

1. Shipment Features (6 features)

- Shipping Mode
- Days for shipment (scheduled)
- Order Item Quantity
- Order Item Product Price
- Order Item Discount Rate
- Sales

2. Market and Logistics Features (4 features)

- Market
- Order Region
- Order Country
- Order State

3. Order and Transaction Features (1 feature)

- Type (Payment / Transaction Type)

4. Temporal and Interaction Features (8 features)

- Order Month

Order Day

- Order Day of Week
- Order Hour
- Order Week of Year
- Is Weekend

Is Night

- Interaction_ShippingMode_OrderHour

Target Variable

- Late_delivery_risk

3.1.2 Explore the Dataset

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery_risk	Category Id	Category Name	Customer City	...	Order Zipcode	Product Card Id	Product Category Id	Product Description	Product Image	Product Name	Product Price	Product Status	shipping date (DateOrders)	Shipping Mode
0	DEBIT	3	4	91.250000	314.640015	Advance shipping	0	73	Sporting Goods	Caguas	...	NaN	1360	73	NaN	http://images.acmesports.sports/Smart+watch	Smart watch	327.75	0	2/3/2018 22:56	Standard Class
1	TRANSFER	5	4	-249.089996	311.359985	Late delivery	1	73	Sporting Goods	Caguas	...	NaN	1360	73	NaN	http://images.acmesports.sports/Smart+watch	Smart watch	327.75	0	1/18/2018 12:27	Standard Class
2	CASH	4	4	-247.779999	309.720001	Shipping on time	0	73	Sporting Goods	San Jose	...	NaN	1360	73	NaN	http://images.acmesports.sports/Smart+watch	Smart watch	327.75	0	1/17/2018 12:06	Standard Class
3	DEBIT	3	4	22.860001	304.809998	Advance shipping	0	73	Sporting Goods	Los Angeles	...	NaN	1360	73	NaN	http://images.acmesports.sports/Smart+watch	Smart watch	327.75	0	1/16/2018 11:45	Standard Class
4	PAYMENT	2	4	134.210007	298.250000	Advance shipping	0	73	Sporting Goods	Caguas	...	NaN	1360	73	NaN	http://images.acmesports.sports/Smart+watch	Smart watch	327.75	0	1/15/2018 11:24	Standard Class

5 rows × 53 columns

Figure 3.1: Preview of the DataCo Smart Supply Chain Dataset

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180519 entries, 0 to 180518
Data columns (total 53 columns):
 #   Column                         Non-Null Count   Dtype
---  ------                         --------------   -----
 0   Type                           180519 non-null  object
 1   Days for shipping (real)       180519 non-null  int64
 2   Days for shipment (scheduled)  180519 non-null  int64
 3   Benefit per order              180519 non-null  float64
 4   Sales per customer             180519 non-null  float64
 5   Delivery Status                180519 non-null  object
 6   Late_delivery_risk             180519 non-null  int64
 7   Category Id                    180519 non-null  int64
 8   Category Name                  180519 non-null  object
 9   Customer City                  180519 non-null  object
 10  Customer Country               180519 non-null  object
 11  Customer Email                 180519 non-null  object
 12  Customer Fname                 180519 non-null  object
 13  Customer Id                    180519 non-null  int64
 14  Customer Lname                 180511 non-null  object
 15  Customer Password              180519 non-null  object
 16  Customer Segment               180519 non-null  object
 17  Customer State                 180519 non-null  object
 18  Customer Street                180519 non-null  object
 19  Customer Zipcode               180516 non-null  float64
 20  Department Id                  180519 non-null  int64
 21  Department Name                180519 non-null  object
 22  Latitude                       180519 non-null  float64
 23  Longitude                      180519 non-null  float64
 24  Market                         180519 non-null  object
 25  Order City                     180519 non-null  object
 26  Order Country                  180519 non-null  object
 27  Order Customer Id              180519 non-null  int64
 28  order date (DateOrders)        180519 non-null  object
 29  Order Id                       180519 non-null  int64
```

```
 30  Order Item Cardprod Id         180519 non-null  int64
 31  Order Item Discount            180519 non-null  float64
 32  Order Item Discount Rate       180519 non-null  float64
 33  Order Item Id                  180519 non-null  int64
 34  Order Item Product Price       180519 non-null  float64
 35  Order Item Profit Ratio        180519 non-null  float64
 36  Order Item Quantity            180519 non-null  int64
 37  Sales                          180519 non-null  float64
 38  Order Item Total               180519 non-null  float64
 39  Order Profit Per Order         180519 non-null  float64
 40  Order Region                   180519 non-null  object
 41  Order State                    180519 non-null  object
 42  Order Status                   180519 non-null  object
 43  Order Zipcode                  24840 non-null   float64
 44  Product Card Id                180519 non-null  int64
 45  Product Category Id            180519 non-null  int64
 46  Product Description            0 non-null       float64
 47  Product Image                  180519 non-null  object
 48  Product Name                   180519 non-null  object
 49  Product Price                  180519 non-null  float64
 50  Product Status                 180519 non-null  int64
 51  shipping date (DateOrders)     180519 non-null  object
 52  Shipping Mode                  180519 non-null  object
dtypes: float64(15), int64(14), object(24)
memory usage: 73.0+ MB
```

Figure 3.2: Information of the Dataset

```
df.shape
```

```
(180519, 53)
```

Figure 3.3: Shape of the Dataset

```
Index(['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)',
       'Benefit per order', 'Sales per customer', 'Delivery Status',
       'Late_delivery_risk', 'Category Id', 'Category Name', 'Customer City',
       'Customer Country', 'Customer Email', 'Customer Fname', 'Customer Id',
       'Customer Lname', 'Customer Password', 'Customer Segment',
       'Customer State', 'Customer Street', 'Customer Zipcode',
       'Department Id', 'Department Name', 'Latitude', 'Longitude', 'Market',
       'Order City', 'Order Country', 'Order Customer Id',
       'order date (DateOrders)', 'Order Id', 'Order Item Cardprod Id',
       'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',
       'Order Item Product Price', 'Order Item Profit Ratio',
       'Order Item Quantity', 'Sales', 'Order Item Total',
       'Order Profit Per Order', 'Order Region', 'Order State', 'Order Status',
       'Order Zipcode', 'Product Card Id', 'Product Category Id',
       'Product Description', 'Product Image', 'Product Name', 'Product Price',
       'Product Status', 'shipping date (DateOrders)', 'Shipping Mode'],
      dtype='object')
```

Figure 3.4: Dataset Attributes (Column Names)

Days for shipping (real)	180519.0	3.497654	1.623722	0.000000	2.000000	3.000000	5.000000	6.000000
Days for shipment (scheduled)	180519.0	2.931847	1.374449	0.000000	2.000000	4.000000	4.000000	4.000000
Benefit per order	180519.0	21.974989	104.433526	-4274.979980	7.000000	31.520000	64.800003	911.799988
Sales per customer	180519.0	183.107609	120.043670	7.490000	104.379997	163.990005	247.399994	1939.989990
Late_delivery_risk	180519.0	0.548291	0.497664	0.000000	0.000000	1.000000	1.000000	1.000000
Category Id	180519.0	31.851451	15.640064	2.000000	18.000000	29.000000	45.000000	76.000000
Customer Id	180519.0	6691.379495	4162.918106	1.000000	3258.500000	6457.000000	9779.000000	20757.000000
Customer Zipcode	180516.0	35921.126914	37542.461122	603.000000	725.000000	19380.000000	78207.000000	99205.000000
Department Id	180519.0	5.443460	1.629246	2.000000	4.000000	5.000000	7.000000	12.000000
Latitude	180519.0	29.719955	9.813646	-33.937553	18.265432	33.144863	39.279617	48.781933
Longitude	180519.0	-84.915675	21.433241	-158.025986	-98.446312	-76.847908	-66.370583	115.263077
Order Customer Id	180519.0	6691.379495	4162.918106	1.000000	3258.500000	6457.000000	9779.000000	20757.000000
Order Id	180519.0	36221.894903	21045.379569	1.000000	18057.000000	36140.000000	54144.000000	77204.000000
Order Item Cardprod Id	180519.0	692.509764	336.446807	19.000000	403.000000	627.000000	1004.000000	1363.000000
Order Item Discount	180519.0	20.664741	21.800901	0.000000	5.400000	14.000000	29.990000	500.000000
Order Item Discount Rate	180519.0	0.101668	0.070415	0.000000	0.040000	0.100000	0.160000	0.250000
Order Item Id	180519.0	90260.000000	52111.490959	1.000000	45130.500000	90260.000000	135389.500000	180519.000000
Order Item Product Price	180519.0	141.232550	139.732492	9.990000	50.000000	59.990002	199.990005	1999.989990
Order Item Profit Ratio	180519.0	0.120647	0.466796	-2.750000	0.080000	0.270000	0.360000	0.500000
Order Item Quantity	180519.0	2.127638	1.453451	1.000000	1.000000	1.000000	3.000000	5.000000
Sales	180519.0	203.772096	132.273077	9.990000	119.980003	199.919998	299.950012	1999.989990
Order Item Total	180519.0	183.107609	120.043670	7.490000	104.379997	163.990005	247.399994	1939.989990

Figure 3.5: Statistical Summary

3.2 Data Preprocessing

Missing values were present in some of the original dataset attributes. These attributes were excluded during feature selection and were therefore not used as inputs to the final prediction model. The initial exploratory feature subset of 31 candidate attributes was systematically refined down to the 19 final model input features strictly available pre-dispatch, completely eliminating target leakage. These 19 final model input features contained no missing values, so imputation was not required.

3.2.1 Data Cleaning

a) Checking for Missing Values

Data preprocessing is a crucial Machine Learning step that prepares the dataset for model training. In SupplyGuard – AI-Powered Shipment Delay Prediction System, the DataCo Smart Supply Chain Dataset is processed by checking missing values, removing duplicates where

necessary, selecting relevant features, encoding categorical variables, and splitting the data into training and testing sets. The 19 final model input features contain no missing values, so missing-value imputation is not required. These steps prepare the data for training and evaluation of the Random Forest and Support Vector Machine (SVM) models for shipment late-delivery-risk prediction.

```
Missing Values:
Product Description    180519
Order Zipcode          155679
Customer Lname              8
Customer Zipcode            3
dtype: int64

Total Missing Values:
336209
```

Figure 3.6: Missing Values in Dataset Attributes

b) Check for Duplicates

Duplicate records were checked and removed from the dataset to avoid repeated information and prevent bias during model training. The analysis showed that there were 0 duplicate records present in the dataset. Therefore, no records were removed during this step, and the dataset retained its original size of 180,519 records with 53 features after duplicate checking.

```
# Checking duplicate records
duplicate_count = df.duplicated().sum()

print("Number of duplicate records:", duplicate_count)

# Removing duplicate records
df = df.drop_duplicates()

# Checking dataset shape after removing duplicates
print("Shape after removing duplicates:", df.shape)
```

```
Number of duplicate records: 0
Shape after removing duplicates: (180519, 53)
```

Figure 3.7: Checking Duplicate Records

c) Outlier Handling

Outlier analysis was performed on the numerical features of the SupplyGuard dataset using the Interquartile Range (IQR) method. Some extreme values were identified in numerical attributes

such as order quantity, product price, discount rate, and sales. These values represent valid variations in supply chain transactions rather than data entry errors. Therefore, the identified outliers were retained without removal or transformation

```
{'Days for shipping (real)': 0,
 'Days for shipment (scheduled)': 0,
 'Benefit per order': 18942,
 'Sales per customer': 1943,
 'Late_delivery_risk': 0,
 'Category Id': 0,
 'Customer Id': 1198,
 'Customer Zipcode': 0,
 'Department Id': 362,
 'Latitude': 9,
 'Longitude': 1414,
 'Order Customer Id': 1198,
 'Order Id': 0,
 'Order Item Cardprod Id': 0,
 'Order Item Discount': 7537,
 'Order Item Discount Rate': 0,
 'Order Item Id': 0,
 'Order Item Product Price': 2048,
 'Order Item Profit Ratio': 17300,
 'Order Item Quantity': 0,
 'Sales': 488,
 'Order Item Total': 1943,
 'Order Profit Per Order': 18942,
 'Order Zipcode': 0,
 'Product Card Id': 0,
 'Product Category Id': 0,
 'Product Description': 0,
 'Product Price': 2048,
 'Product Status': 0}
```

Figure 3.8: Outliers Identified in Dataset

d) One-Hot Encoding

Categorical encoding is used to convert text data into numerical values that machine learning algorithms can understand. In SupplyGuard, features such as Shipping Mode, Payment Method, Market, Order Region, and Order State are converted into separate binary columns using One-Hot Encoding, with values of 0 or 1. This helps the Random Forest and SVM models process categorical data without assuming any order or ranking between categories.

```
SupplyGuard Categorical Features — One-Hot Encoding:

[1] Shipping Mode (4 binary columns):
    Standard Class       ->  [1, 0, 0, 0]
    First Class          ->  [0, 1, 0, 0]
    Second Class         ->  [0, 0, 1, 0]
    Same Day             ->  [0, 0, 0, 1]

[2] Type / Payment Method (4 binary columns):
    DEBIT                ->  [1, 0, 0, 0]
    TRANSFER             ->  [0, 1, 0, 0]
    PAYMENT              ->  [0, 0, 1, 0]
    CASH                 ->  [0, 0, 0, 1]

[3] High-Cardinality Geographic & Interaction Features:
    Market (5 sectors)          ->  5 binary columns  (Pacific Asia, Europe...)
    Order Region (23 regions)   ->  23 binary columns (Central America...)
    Order Country (135+ nations)->  135+ binary cols  (United States, France...)
    Order State (1,100+ states) ->  1,100+ binary cols(California, Texas...)
```

Figure 3.9: One-Hot Encoding Transformation of SupplyGuard Categorical Features

3.2.2 Analysis of Feature Variables

a) Feature Variables

The feature variables were analyzed to understand their characteristics, data types, distributions, missing values, and their relevance for Machine Learning model development. This analysis helps identify the important attributes, detect inconsistencies, and ensure that the dataset is suitable for further preprocessing and model training. The features were categorized into numerical and categorical variables based on their data types.

```
numerical_features = df.select_dtypes(include=['int64', 'float64']).columns.tolist()

categorical_features = df.select_dtypes(include=['object']).columns.tolist()


print("Numerical Feature Variables:\n")
for i, feature in enumerate(numerical_features, 1):
    print(i, ".", feature)


print("\n\nCategorical Feature Variables:\n")
for i, feature in enumerate(categorical_features, 1):
    print(i, ".", feature)
```

```
Numerical Feature Variables:

1 . Days for shipping (real)
2 . Days for shipment (scheduled)
3 . Benefit per order
4 . Sales per customer
5 . Late_delivery_risk
6 . Category Id
7 . Customer Id
8 . Customer Zipcode
9 . Department Id
10 . Latitude
11 . Longitude
12 . Order Customer Id
13 . Order Id
14 . Order Item Cardprod Id
15 . Order Item Discount
16 . Order Item Discount Rate
17 . Order Item Id
18 . Order Item Product Price
19 . Order Item Profit Ratio
20 . Order Item Quantity
21 . Sales
22 . Order Item Total
23 . Order Profit Per Order
24 . Order Zipcode
25 . Product Card Id
26 . Product Category Id
27 . Product Description
28 . Product Price
29 . Product Status


Categorical Feature Variables:

1 . Type
2 . Delivery Status
3 . Category Name
4 . Customer City
5 . Customer Country
6 . Customer Email
7 . Customer Fname
8 . Customer Lname
9 . Customer Password
10 . Customer Segment
11 . Customer State
12 . Customer Street
13 . Department Name
14 . Market
15 . Order City
16 . Order Country
17 . order date (DateOrders)
18 . Order Region
19 . Order State
20 . Order Status
21 . Product Image
22 . Product Name
23 . shipping date (DateOrders)
24 . Shipping Mode
```

Figure 3.10: Feature Variables Selected for Model Development

b) Summary of feature variables

```
feature_summary = pd.DataFrame({
    'Feature Name': df.columns,
    'Data Type': df.dtypes.values,
    'Missing Values': df.isnull().sum().values,
    'Unique Values': df.nunique().values
})

feature_summary
```

	Feature Name	Data Type	Missing Values	Unique Values
0	Type	object	0	4
1	Days for shipping (real)	int64	0	7
2	Days for shipment (scheduled)	int64	0	4
3	Benefit per order	float64	0	21998
4	Sales per customer	float64	0	2927
5	Delivery Status	object	0	4
6	Late_delivery_risk	int64	0	2
7	Category Id	int64	0	51
8	Category Name	object	0	50
9	Customer City	object	0	563
10	Customer Country	object	0	2
11	Customer Email	object	0	1
12	Customer Fname	object	0	782
13	Customer Id	int64	0	20652
14	Customer Lname	object	8	1109
15	Customer Password	object	0	1
16	Customer Segment	object	0	3
17	Customer State	object	0	46
18	Customer Street	object	0	7458
19	Customer Zipcode	float64	3	995
20	Department Id	int64	0	11
21	Department Name	object	0	11
22	Latitude	float64	0	11250
23	Longitude	float64	0	4487
24	Market	object	0	5
25	Order City	object	0	3597

26	Order Country	object	0	164
27	Order Customer Id	int64	0	20652
28	order date (DateOrders)	object	0	65752
29	Order Id	int64	0	65752
30	Order Item Cardprod Id	int64	0	118
31	Order Item Discount	float64	0	1017
32	Order Item Discount Rate	float64	0	18
33	Order Item Id	int64	0	180519
34	Order Item Product Price	float64	0	75
35	Order Item Profit Ratio	float64	0	162
36	Order Item Quantity	int64	0	5
37	Sales	float64	0	193
38	Order Item Total	float64	0	2927
39	Order Profit Per Order	float64	0	21998
40	Order Region	object	0	23
41	Order State	object	0	1089
42	Order Status	object	0	9
43	Order Zipcode	float64	155679	609
44	Product Card Id	int64	0	118
45	Product Category Id	int64	0	51
46	Product Description	float64	180519	0
47	Product Image	object	0	118
48	Product Name	object	0	118
49	Product Price	float64	0	75
50	Product Status	int64	0	1
51	shipping date (DateOrders)	object	0	63701
52	Shipping Mode	object	0	4

Figure 3.11: Summary of Feature Variables

3.2.3 Analysis of Class Variables

The class variable represents the output that the Machine Learning model aims to predict. In SupplyGuard, the target variable is Late_delivery_risk, where class 0 represents no late-delivery risk and class 1 represents late-delivery risk. The class distribution is analyzed to understand the representation of each class and its suitability for binary classification.

```
# Define the class variable (target variable)
class_variable = 'Delivery Status'   # Change this if your target column is different

# Class Variable Summary

class_summary = pd.DataFrame({
    'Class Name': df[class_variable].value_counts().index,
    'Frequency': df[class_variable].value_counts().values,
    'Percentage (%)': (df[class_variable].value_counts(normalize=True) * 100).round(2).values
})

display(class_summary)
```

	Class Name	Frequency	Percentage (%)
0	Late delivery	98977	54.83
1	Advance shipping	41592	23.04
2	Shipping on time	32196	17.84
3	Shipping canceled	7754	4.30

Figure 3.12: Summary of Class Variable Distribution

3.3 Data Visualization

"Data visualization is performed to represent the patterns, relationships, and distribution of data using graphical techniques. It helps in understanding the dataset more effectively by identifying trends, outliers, class distribution, and important relationships between variables. Various visualization methods such as count plots, histograms, box plots, and correlation heatmaps are used to analyze the SupplyGuard dataset before model development."

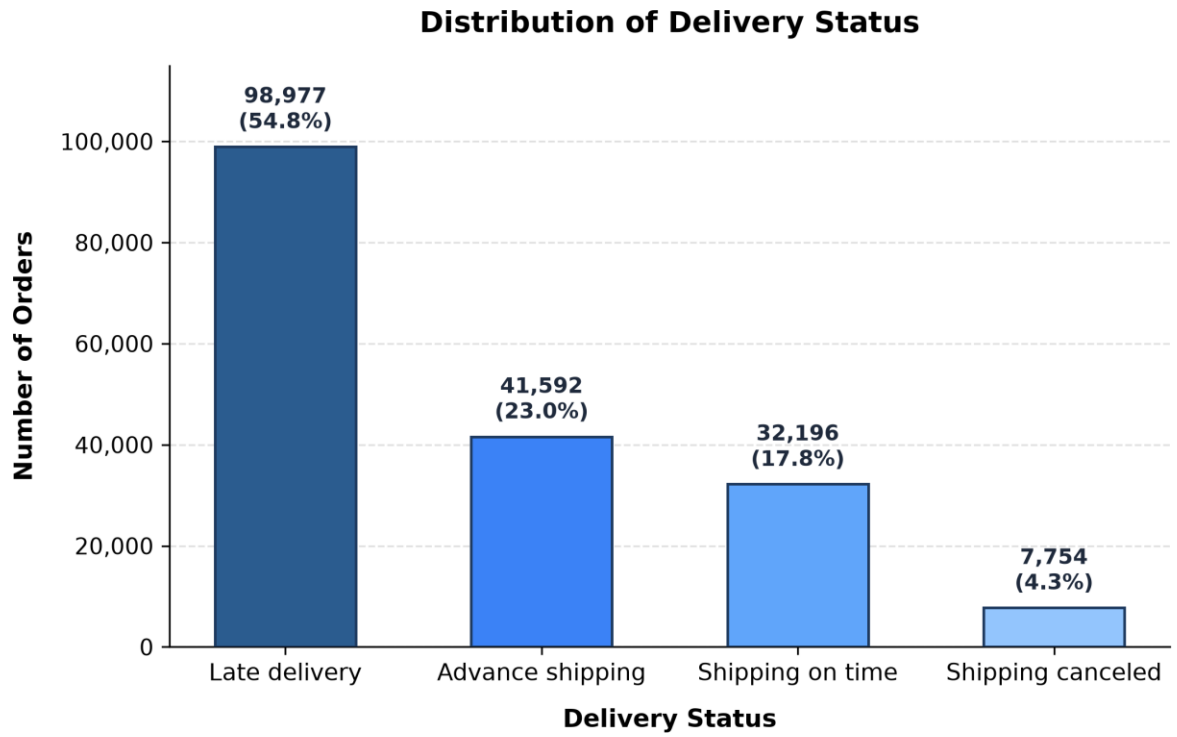


Figure 3.13: Data Visualization of Key Features

3.4 Analysis of Architecture and Algorithms

The SupplyGuard system uses Machine Learning and supply chain data analysis to predict shipment late-delivery risk. The system follows a structured process in which shipment data is analyzed, relevant features are selected and engineered, and the data is preprocessed for model training. The processed data is then provided to classification algorithms to predict late-delivery risk.

Two supervised Machine Learning algorithms, Random Forest and Support Vector Machine (SVM), are evaluated for shipment late-delivery-risk prediction. Random Forest combines

multiple decision trees to produce classification results, while SVM identifies a decision boundary between different classes using an RBF kernel. Both models are evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Based on the experimental results, Random Forest is used as the final prediction model.

3.4.1 Detailed Study Report of the Proposed Algorithm(s)

The SupplyGuard system uses supervised Machine Learning algorithms to predict shipment late-delivery risk. The selected dataset features are preprocessed and categorical variables are converted using One-Hot Encoding before model training. Two classification algorithms, Random Forest and Support Vector Machine (SVM), are evaluated using historical shipment data.

The system consists of three major components: Data Preprocessing, Feature Selection and Engineering, and Machine Learning Classification. Data preprocessing prepares the selected features for modelling, while feature selection and engineering identify the relevant variables and derive temporal and interaction features. The processed features are then used to train and evaluate the Random Forest and SVM models.

Random Forest is used as the final prediction model after comparing its performance with SVM using the selected evaluation metrics.

3.4.1.1 Random Forest

Random Forest is a supervised Machine Learning algorithm based on ensemble learning. It combines multiple decision trees to improve classification performance and reduce the risk of overfitting. Each tree is trained using randomly selected samples and features, and the outputs of the trees are combined to produce the final classification.

In SupplyGuard, Random Forest is used to classify shipments into late-delivery-risk and non-late-delivery-risk classes. It can model non-linear relationships between the selected shipment, customer, product, market, geographical, and temporal features. The algorithm also provides feature importance values for analyzing the contribution of the input features.

The final Random Forest model uses 200 decision trees with a tuned decision threshold of 0.41, with min_samples_leaf = 2 and random_state = 42.

Steps involved in Random Forest:

1. Selecting multiple random samples from the training dataset.

2. Constructing individual decision trees using selected samples.
3. Generating predictions from each decision tree.
4. Combining all tree outputs to produce the final prediction.

The Random Forest algorithm can be mathematically represented as:

Prediction = Majority Vote ($Tree_1$, $Tree_2$, $Tree_3$, …, $Tree_n$)

where each tree contributes independently to the final classification decision.

For classification, the Random Forest combines the predictions of individual decision trees to determine the final class. The final SupplyGuard model also produces a probability score, which is compared with a threshold of 0.41 to determine the final late-delivery-risk prediction.

The main advantages of Random Forest include:

- Combines multiple decision trees for classification.
- Reduced risk of overfitting compared to a single decision tree.
- Handles numerical and encoded categorical features effectively.
- Provides feature importance values.
- Can model non-linear relationships in the data.

3.4.1.2 Support Vector Machine (SVM) Algorithm

Support Vector Machine (SVM) is a supervised Machine Learning algorithm commonly used for classification problems. It identifies a decision boundary that separates different classes while maximizing the margin between them. In SupplyGuard, SVM is used to classify shipments into late-delivery-risk and non-late-delivery-risk classes.

An RBF (Radial Basis Function) kernel is used because it allows the model to represent non-linear relationships in the transformed feature space. The SVM model uses C = 1.0, gamma = scale, and probability estimation enabled.

Due to the higher computational cost of SVM on the complete training data, the SVM experiment was trained using a stratified subset of 30,000 records from the training data. The model was then evaluated using the same test set used for Random Forest. The working process of SVM involves:

1. Preprocessing and scaling the input features.

2. Transforming the data into a suitable feature space.
3. Identifying important data points called support vectors.
4. Determining an optimal decision boundary between the classes.
5. Using the learned boundary to classify new shipment records.

The main objective of SVM is to find an optimal decision boundary that separates the different classes while maximizing the margin between them. For the SupplyGuard system, the decision boundary is used to distinguish between shipments with late-delivery risk and shipments without late-delivery risk.

The important features of SVM include:

- Effective classification of shipment delay risk into two classes.
- Ability to model non-linear relationships using kernel functions.
- RBF kernel support for handling complex patterns in shipment data.

- Effective performance with high-dimensional feature spaces after encoding.
- Uses support vectors to determine the optimal classification boundary.
- Provides a suitable alternative for comparison with the Random Forest algorithm.

3.4.1.3 Comparison of Proposed Algorithms

Random Forest and Support Vector Machine (SVM) are both supervised machine learning algorithms that provide effective solutions for classification problems. However, they differ in their learning approach, decision-making process, computational requirements, and performance characteristics

Feature	**Random Forest**	**SVM**
Learning Method	Ensemble learning using multiple decision trees	Margin-based classification method
Model Construction	Multiple trees are built independently	An optimal decision boundary is determined from the training data

Decision Process	Combines predictions from multiple trees using majority voting	Classifies data based on the learned decision boundary
Training Speed	Suitable for large datasets and supports parallel processing	Can require more computational time as dataset size increases
Overfitting Control	Reduces overfitting by combining multiple decision trees	Controls model complexity using margin maximization and regularization
Feature Importance	Provides feature-importance information	Does not directly provide tree-based feature importance
Dataset Suitability	Works well with large and complex structured datasets	Suitable for classification with high-dimensional feature spaces
SupplyGuard Training	Trained using full training dataset with 200 trees (calibrated threshold 0.41)	Trained using a stratified subset of 30,000 training records

3.4.2 Activity Diagram to Represent the Flow of the Algorithm(s)

The activity diagram illustrates the workflow of the SupplyGuard system, from shipment data processing to final late-delivery-risk prediction. The process includes data preprocessing, feature selection, feature engineering, One-Hot Encoding, and train-test splitting before the data is provided to the Random Forest and SVM models for evaluation.

The performance of the models is evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Based on the experimental results, Random Forest (200 decision trees, decision threshold 0.41) is used as the final prediction model. The trained model is integrated with the web interface, where shipment information is entered and the system generates a late-delivery probability and corresponding risk prediction.

The diagram provides a clear representation of the sequential interaction between data preprocessing, feature engineering, Machine Learning model evaluation, prediction generation, and result display on the dashboard.

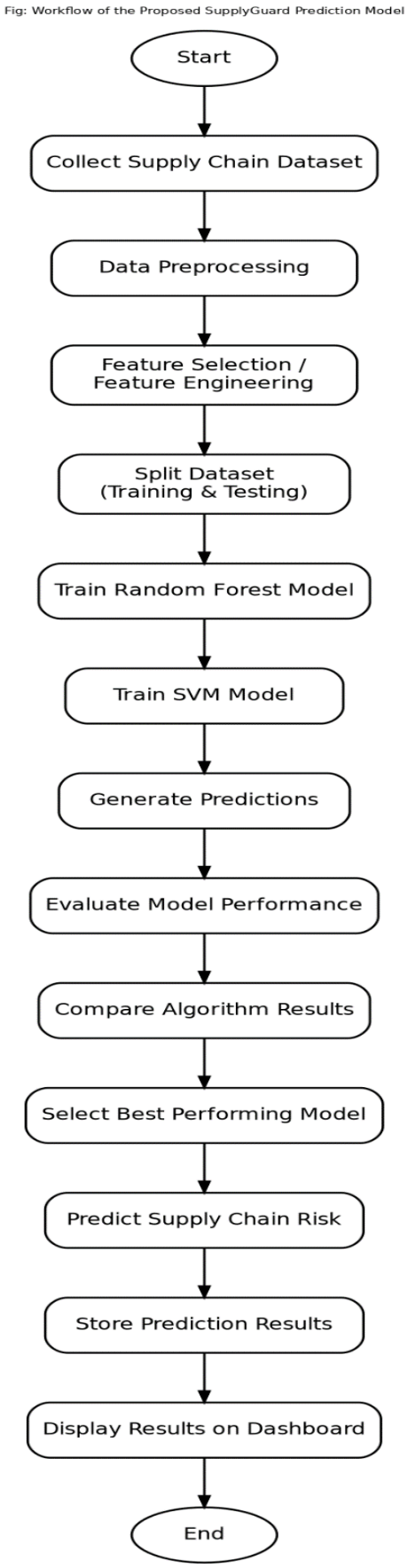


Figure 3.14: Activity Diagram of the Proposed SupplyGuard System

3.4.3 Step-by-Step Explanation about the Working of the Algorithm(s)

The proposed SupplyGuard system processes supply chain data through a sequence of data preprocessing, feature analysis, and Machine Learning techniques to predict potential shipment delays and improve supply chain decision-making. The complete working of the proposed algorithms, Random Forest and SVM, is described below.

Step 1: Collect Supply Chain Data

The process begins with the DataCo Smart Supply Chain Dataset, which contains historical supply chain records related to customers, products, orders, shipments, markets, and geographical information. The dataset is used as the source for developing the shipment late-delivery-risk prediction model.

Step 2: Load Dataset into the System

The dataset is loaded into the Python environment for analysis and model development. The structure, attributes, data types, missing values, duplicate records, and statistical characteristics of the dataset are examined before model preparation.

Step 3: Perform Data Preprocessing

The dataset is prepared for Machine Learning by checking missing values and duplicate records, excluding unsuitable attributes, selecting relevant features, and converting categorical variables into numerical form using One-Hot Encoding. The final selected features do not require missing-value imputation.

Step 4: Perform Feature Selection and Transformation

Relevant features are selected for shipment late-delivery-risk prediction. The final model uses 19 selected features consisting of categorical and numerical variables. Temporal features are derived from the order date (DateOrders) attribute, including order month, day, hour, day of week, weekend, and other calendar-related features. An interaction feature, Interaction_ShippingMode_OrderHour, is also created by combining Shipping Mode and Order Hour

Step 5: Split Dataset into Training and Testing Data

The processed dataset is divided into training and testing subsets using an 80:20 stratified split. The training set contains 144,415 records, while the test set contains 36,104 records. The

training data is used for model development, and the test data is used to evaluate model performance on unseen records.

Step 6: Apply Random Forest and SVM Algorithms

The processed training data is used to evaluate the Random Forest and SVM classification algorithms. Random Forest combines the predictions of multiple decision trees to perform classification. The final Random Forest model uses 200 decision trees with a tuned decision threshold of 0.41. SVM determines a decision boundary between the two classes and uses an RBF kernel to model non-linear relationships. Due to its higher computational cost, the SVM experiment is trained using a stratified subset of 30,000 records from the training data.

Step 7: Generate Risk Prediction

The trained models generate predictions for shipment late-delivery risk. The final Random Forest model also produces a probability score representing the predicted likelihood of late delivery. In SupplyGuard, the probability is compared with a threshold of **0.41**. A probability of 0.41 or higher is classified as late-delivery risk, while a probability below 0.41 is classified as no late-delivery risk

Step 8: Evaluate and Compare Model Performance

Random Forest and SVM are evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Their results are compared based on the experimental configurations used in the project. Random Forest is selected as the final prediction model based on its measured performance.

Step 9: Store Prediction Results

The trained Random Forest model and its preprocessing components are saved as a Machine Learning pipeline. The saved pipeline allows new shipment information entered through the webpage to undergo the same preprocessing steps used during model development before generating a prediction.

Step 10: Display Results on the Dashboard

The webpage accepts the required shipment information as input and passes it to the saved SupplyGuard model pipeline. The system generates the late-delivery probability and displays the corresponding late-delivery-risk prediction to the user.

Step 11: Complete the Process

After displaying the prediction result, the process is completed. The webpage can then be used to enter another shipment record and generate a new late-delivery-risk prediction.

3.5 Project Pipeline

The SupplyGuard pipeline covers dataset collection, analysis, feature engineering, preprocessing, model training, evaluation, and prediction. It provides a systematic workflow for predicting shipment delay risks using machine learning.

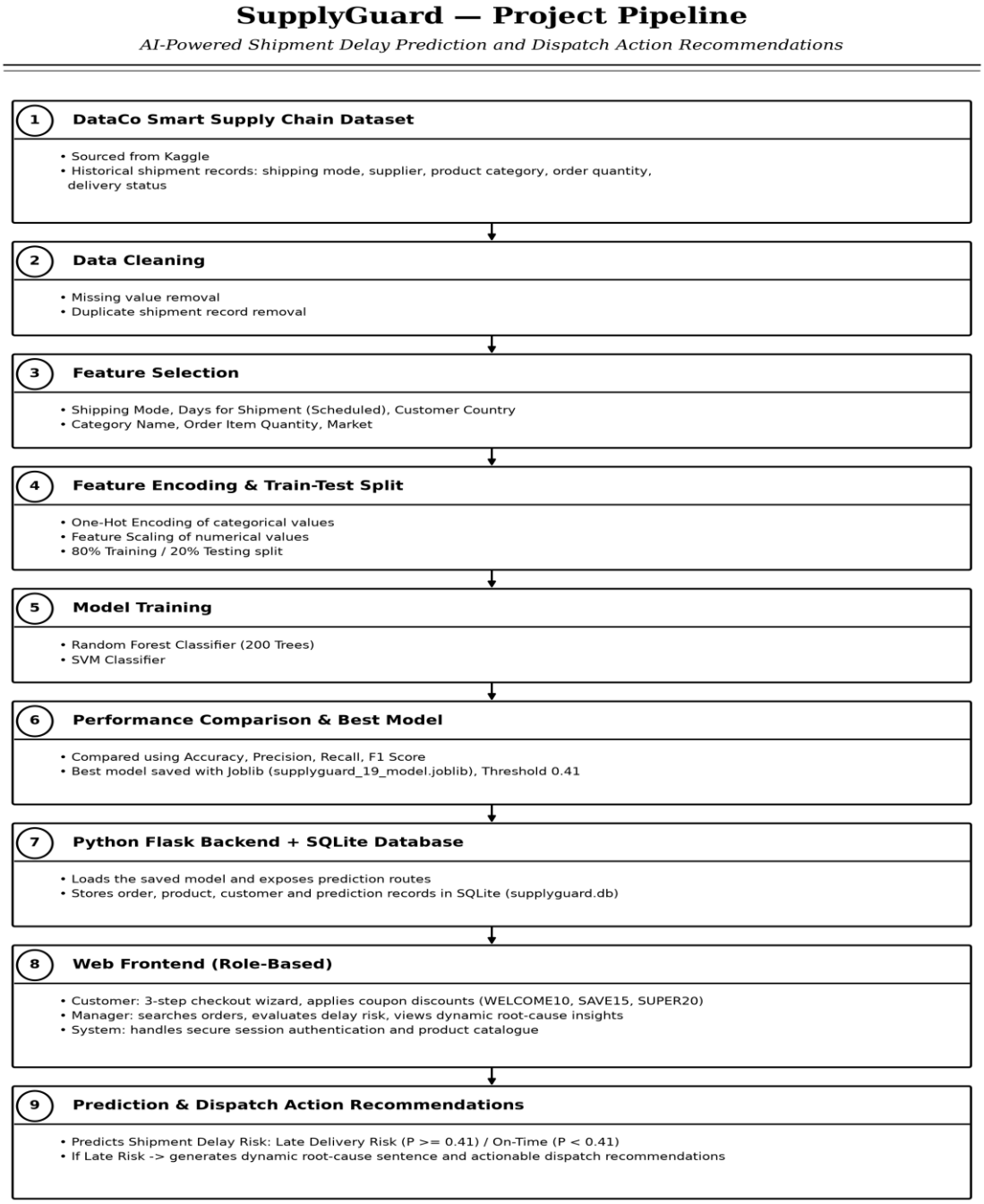


Figure 3.15: Project Pipeline of the Proposed SupplyGuard System

Level 1: Dataset Collection

The project begins with the DataCo Smart Supply Chain Dataset, which contains historical records related to customers, products, orders, shipments, markets, and geographical information. Sourced from Kaggle, the dataset provides multi-domain supply chain features used to develop and evaluate the Machine Learning models for shipment late-delivery-risk prediction.

Level 2: Data Preprocessing

The dataset is analyzed to understand its structure, attributes, data types, missing values, duplicate records, and statistical characteristics. Crucially, all post-dispatch data leakage columns (such as 'Days for shipping (real)' and delivery status) are excluded so the model learns strictly from pre-dispatch variables available at order entry.

Level 3: Feature Selection and Engineering

Relevant features for shipment late-delivery-risk prediction are selected from the dataset, focusing on 19 pre-dispatch features. Temporal features are derived from the order date attribute, and an interaction feature is created by combining Shipping Mode and Order Hour (Interaction_ShippingMode_OrderHour) to capture operational dispatch time dependencies.

Level 4: Feature Encoding & Train-Test Split

Categorical variables are converted into numerical form using One-Hot Encoding, while continuous numerical attributes are scaled. The dataset is divided into training and testing subsets using an 80:20 stratified split (144,415 training records and 36,104 holdout test records) to maintain class balance.

Level 5: Model Training

The processed training data is used to train the proposed Random Forest (200 decision trees) and Support Vector Machine (SVM with RBF kernel) classification models to compare ensemble decision trees against maximum-margin baselines.

Level 6: Performance Comparison & Best Model

The trained models are evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Random Forest is selected as the champion model, achieving 83.89% Accuracy and 91.18% ROC-AUC. The decision threshold is tuned to 0.41 (yielding 88.31% recall on delayed shipments), and the complete pipeline is serialized with Joblib into 'saved_model/supplyguard_19_model.joblib'.

Level 7: Python Flask Backend + SQLite Database

The backend application is implemented using Python Flask with Werkzeug WSGI and Jinja2 templating. An SQLite relational database ('supplyguard.db') manages users, products, orders, and order_items tables, paired with the Cascading Geographic Routing Engine.

Level 8: Web Frontend (Role-Based)

The user interface incorporates role-based access control. Authenticated customers access a 3-step checkout wizard with coupon code validation (WELCOME10, SAVE15, SUPER20), while logistics managers access a dedicated logistics cockpit with real-time order lookup and risk assessment triggers.

Level 9: Prediction & Dispatch Action Recommendations

When an order is assessed, the system generates a late-delivery probability compared against the 0.41 threshold. The user views the predicted late-delivery probability and corresponding risk classification, accompanied by dynamic root-cause insights and actionable dispatch recommendations.

These technologies are widely available and support data preprocessing, feature engineering, One-Hot Encoding, model training, evaluation, model saving, and prediction. The system can be developed and executed using standard computer hardware without specialized infrastructure. Therefore, the proposed SupplyGuard system is technically feasible.

3.6 Feasibility Analysis

Feasibility analysis evaluates whether a proposed system can be successfully developed and implemented using the available resources, technologies, budget, and operating environment. For the proposed SupplyGuard system, the feasibility study considers technical, economic, and operational aspects to determine whether the system can effectively predict shipment late-delivery risk while remaining practical and user-friendly.

3.6.1 Technical Feasibility

Technical feasibility evaluates whether the required technologies, hardware, software, and development tools are available to implement the proposed system. SupplyGuard is developed using Python and Machine Learning libraries such as Pandas, NumPy, Scikit-learn, Matplotlib, and Joblib. The system uses Random Forest and Support Vector Machine (SVM) for shipment late-delivery-risk prediction and provides a web-based interface for entering shipment information and displaying prediction results.

These technologies are widely available and support data preprocessing, feature engineering, One-Hot Encoding, model training, evaluation, model saving, and prediction. The system can be developed and executed using standard computer hardware without specialized infrastructure. Therefore, the proposed SupplyGuard system is technically feasible.

3.6.2 Economic Feasibility

Economic feasibility assesses whether the proposed system can be developed and implemented within a reasonable budget. SupplyGuard primarily uses open-source programming languages, Machine Learning libraries, and freely available development tools, reducing the need for software licensing costs

The system can be developed using standard computer hardware without requiring specialized infrastructure. The DataCo Smart Supply Chain Dataset is available for use in the project, further reducing data acquisition costs. Therefore, the overall development cost is relatively low, making the proposed SupplyGuard system economically feasible

3.6.3 Operational Feasibility

Operational feasibility determines whether the proposed system can function effectively within its intended environment and satisfy its basic user requirements. SupplyGuard provides a simple web-based interface through which users can enter the required shipment information and obtain a late-delivery-risk prediction.

The system processes the entered information using the saved Machine Learning pipeline and Random Forest model and displays the late-delivery probability and corresponding risk classification. The interface requires only the relevant shipment information as input and presents the prediction result in a simple format. Therefore, the proposed SupplyGuard system is operationally feasible.

3.7 System Environment

The system environment specifies the software tools, Machine Learning libraries, and hardware resources required for the development and implementation of the proposed SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System. The project uses standard software tools and computing resources for supply chain data analysis, feature engineering, data preprocessing, Machine Learning model development, evaluation, and web-based prediction.

3.7.1 Software Environment

The software environment consists of Python and commonly used open-source libraries for data analysis, preprocessing, Machine Learning, visualization, and model deployment. Visual Studio Code is used for project implementation and development. Python libraries such as Pandas and NumPy support dataset handling and numerical operations, while Scikit-learn provides the preprocessing, Random Forest, SVM, and evaluation components. Matplotlib is used for data visualization, and Joblib is used to save and load the trained Machine Learning pipeline.

Software / Tool	**Purpose**
Python	Programming language used for Machine Learning implementation
Visual Studio Code	Code development and project implementation
Pandas	Dataset handling and data analysis
NumPy	Numerical and array-based computations
Scikit-learn	One-Hot Encoding, Random Forest, SVM, data splitting, and model evaluation
Matplotlib	Data visualization and model-result visualization
Joblib	Saving and loading the trained Machine Learning pipeline
Web Interface	Accepting shipment information and displaying prediction results

3.7.2 Hardware Environment

The proposed SupplyGuard system does not require specialized hardware for its development or operation. A standard laptop or desktop computer with sufficient processing capability, memory, and storage can be used for supply chain data analysis, feature engineering, Machine Learning model training, testing, and web interface development.

Hardware Component	**Minimum Requirement**

Processor	Dual-Core 2.0 GHz or higher (Intel Core i5 recommended for model training)
RAM	4 GB minimum (8 GB recommended for dataset preprocessing and model training)
Storage	2 GB available free disk space (on a system with 128 GB / 256 GB total device storage)
Display	Standard display (1366 x 768 or higher)
Internet Connection	Stable internet connection for required libraries / Localhost execution

4. System Design

4.1 Models Building

Model building is the central phase of the proposed SupplyGuard shipment late-delivery-risk prediction system. In this phase, the processed DataCo Smart Supply Chain Dataset is used to develop Machine Learning models for classifying shipments into late-delivery-risk and no-late-delivery-risk classes. Random Forest and Support Vector Machine (SVM) are implemented and evaluated to compare their prediction performance.

The model-building process consists of implementation, model planning, training, and testing. The dataset is divided into training and testing sets, the models are trained using the prepared features, and their performance is evaluated using Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix.

4.1.1 Implementation Code

The Machine Learning component of the SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System is implemented in Python using the Scikit-learn library for feature transformation, model training, and evaluation. The system uses Random Forest as the final prediction algorithm and Support Vector Machine (SVM) as the comparison algorithm. The selected shipment features are processed using a Scikit-learn preprocessing pipeline, where categorical variables are converted into numerical representations using One-Hot Encoding, while numerical features are passed directly to the Machine Learning models. The important parts of the implementation are shown below.

```
preprocessor = ColumnTransformer(
transformers=[
("cat", OneHotEncoder(handle_unknown="ignore"),
categorical_features),
("num", "passthrough",
numerical_features)
]
)
```

The ColumnTransformer applies different preprocessing operations according to the type of feature. The categorical features are transformed using One-Hot Encoding, with handle_unknown="ignore" allowing the pipeline to process previously unseen categorical values during prediction. The numerical features are passed through without scaling because the final Random Forest model does not require feature standardization.

The final model uses 19 pre-dispatch features, consisting of 7 categorical features and 12 numerical features. After One-Hot Encoding, these features are transformed into a clean numerical feature matrix that completely eliminates post-dispatch data leakage.

The preprocessing component is combined with the Random Forest classifier in a Scikit-learn Pipeline so that the same feature transformation is automatically applied during training, testing, and prediction.

```
rf_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        n_estimators=200,
        min_samples_leaf=2,
        random_state=42,
        n_jobs=-1
    ))
])
```

The above implementation creates the final Random Forest model. The classifier consists of 200 decision trees with a tuned decision threshold of 0.41, with min_samples_leaf=2 used to control the minimum number of samples required at a leaf node. random_state=42 ensures reproducibility, while n_jobs=-1 allows the available processor cores to be used during training.

The SVM model is implemented as a comparison model using an RBF kernel.

```
svm_model = SVC(
    kernel="rbf",
    C=1.0,
    gamma="scale",
```

```
    probability=True,
    random_state=42
)
```

The SVM uses an RBF (Radial Basis Function) kernel to model non-linear relationships between the transformed input features. The parameter C=1.0 controls the regularization, while gamma="scale" determines the kernel coefficient automatically from the input data. Probability estimation is enabled so that probability-based predictions can be obtained.

Because SVM requires considerably more computational time when applied to the complete training dataset, the SVM experiment is trained using a stratified subset of 30,000 records from the training data. The Random Forest model, in contrast, is trained using the complete training dataset of 144,415 records.

Model	Parameter	Value
Random Forest	n_estimators	200
Random Forest	min_samples_leaf	2
Random Forest	max_depth	None (unlimited)
Random Forest	Decision threshold	0.41
Random Forest	random_state	42
SVM	kernel	rbf
SVM	C	1.0
SVM	gamma	scale
SVM	probability	True
SVM	random_state	42

Table 4.1: Model Hyperparameter Configuration

The Machine Learning pipeline ensures that preprocessing is learned from the training data and then consistently applied to the testing data and new shipment records. This prevents

differences between the data preparation used during model training and the data preparation used during prediction.

For prediction, the saved SupplyGuard pipeline receives the shipment information entered through the web interface. The input is converted into the same feature structure used during model training, and the trained Random Forest model generates a probability of late delivery.

```
prediction_probability = model.predict_proba(input_data)[0][1]

if prediction_probability >= 0.41:

    prediction = 1

else:

    prediction = 0
```

The predict_proba() function generates the probability associated with the late-delivery-risk class. In SupplyGuard, a threshold of 0.41 is used to convert the probability into the final binary prediction. A probability of 0.41 or higher is classified as Late-Delivery Risk, while a probability below 0.41 is classified as No Late-Delivery Risk.

The trained model, preprocessing components, selected feature information, threshold, and evaluation results are saved for later use by the web interface. Therefore, when a user enters new shipment information, the saved Machine Learning pipeline performs the required preprocessing automatically and generates the late-delivery probability and corresponding risk classification.

4.1.2 Model Planning (Training/Testing Split of the Dataset)

Model planning involves dividing the prepared dataset into separate subsets so that the performance of the Machine Learning models can be measured on shipment records that were not used during training. The DataCo Smart Supply Chain Dataset contains 180,519 records, with Late_delivery_risk as the target variable. The target contains two classes: 0 (No Late-Delivery Risk) and 1 (Late-Delivery Risk). The dataset is divided into 80% training data and 20% testing data using stratified splitting.

Dataset Split	Number of Records	Percentage
Training Set	144,415	80%

Dataset Split	Number of Records	Percentage
Testing Set	36,104	20%
Total Records	**180,519**	**100%**

The following code is used to divide the dataset into training and testing sets.

```
X = df[final_features]

y = df["Late_delivery_risk"]

X_train, X_test, y_train, y_test = train_test_split(

X,

y,

test_size=0.2,

random_state=42,

stratify=y

)
```

The train_test_split() function divides the dataset into training and testing subsets. The test_size=0.2 parameter reserves 20% of the records for testing, while random_state=42 makes the split reproducible. The stratify=y parameter maintains the proportion of the two late-delivery-risk classes in both subsets.

The complete dataset contains 81,542 records (45.17%) belonging to class 0 and 98,977 records (54.83%) belonging to class 1. Stratified splitting preserves this class distribution approximately in both the training and testing datasets.

The training dataset containing 144,415 records is used to develop the Random Forest model and conduct the SVM experiment. The testing dataset containing 36,104 records is kept separate from model training and is used to evaluate the performance of both algorithms on unseen shipment records. The Random Forest is trained using the complete training dataset, while the SVM experiment uses a stratified subset of 30,000 records from the training data because of its higher computational cost.

4.1.3 Training – Build the Model Using the Train Dataset

The training phase builds the proposed Machine Learning classifiers using the training dataset. The input features are stored in X_train, while the corresponding target values of Late_delivery_risk are stored in y_train. The Random Forest model is trained using all 144,415 training records, while the SVM experiment is trained using a stratified subset of 30,000 records from the training data because of its higher computational cost.

The Random Forest model is trained through the complete preprocessing pipeline, which applies One-Hot Encoding to the categorical features before passing the transformed data to the classifier

rf_pipeline.fit(X_train, y_train)

The final Random Forest model consists of 200 decision trees with a tuned decision threshold of 0.41. Each tree is constructed using randomly selected samples and features from the training data, and the predictions from the individual trees are combined to produce the classification result. The model uses min_samples_leaf=2, random_state=42, and n_jobs=-1.

The SVM model is trained separately using a stratified subset of 30,000 records from the training dataset

svm_model.fit(X_train_svm, y_train_svm)

The SVM uses an RBF kernel to identify a non-linear decision boundary between the two target classes. The model uses C=1.0, gamma="scale", and probability estimation enabled.

The preprocessing pipeline is fitted using the training data so that the One-Hot Encoding categories are learned from the training records. The same fitted preprocessing is then applied to the test data and new shipment records. This ensures consistency between the data used for model training and the data used for prediction.

The fit() method learns the relationship between the selected shipment features and the target variable, Late_delivery_risk. After training, the models are ready to generate predictions for the held-out test dataset.

4.1.4 Testing – Evaluate the Performance of the Model Using the Test Dataset

After training, both Machine Learning models are evaluated using the 36,104 test records, which were not used during model training. The predictions generated by the models are compared with the actual Late_delivery_risk values stored in y_test.

The following code is used to generate predictions from the Random Forest model:

```
rf_preds = rf_pipeline.predict(X_test)

rf_proba = rf_pipeline.predict_proba(X_test)[:, 1]
```

The SVM model is evaluated using the same held-out test dataset:

```
svm_preds = svm_model.predict(X_test_transformed)

svm_proba = svm_model.predict_proba(X_test_transformed)[:, 1]
```

5. Results and Discussions

5.1 Comparison of Models

The performance of the Random Forest and Support Vector Machine (SVM) models was compared using the same held-out test dataset of 36,104 shipment records. The models were evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC to measure their ability to classify shipments according to Late_delivery_risk. Table 5.1 presents the performance comparison of the two models.

Evaluation Metric	Random Forest	SVM
Accuracy	83.89%	72.42%
Precision	83.31%	78.47%
Recall	88.31%	68.50%
F1-Score	85.73%	73.15%
ROC-AUC	91.18%	79.21%

Table 5.1: Performance Comparison of Random Forest and SVM on the Test Set

The Random Forest model obtained an Accuracy of 83.89% and an F1-Score of 85.73% on the test dataset using 19 pre-dispatch features and a calibrated decision threshold of 0.41. In comparison, the SVM model achieved an Accuracy of 72.42% and an F1-Score of 73.15%. Random Forest demonstrated superior predictive performance across all evaluated metrics.

The difference in Accuracy between the two models is 11.47 percentage points, while the difference in F1-Score is 12.58 percentage points. Random Forest also achieved a higher Recall of 88.31% compared to 68.50% for SVM, indicating that Random Forest identified a significantly higher proportion of shipments with late delivery risk. The ROC-AUC score of Random Forest was 91.18%, compared to 79.21% for SVM, confirming superior class separability across different classification thresholds.

However, the comparison should be interpreted according to the experimental configurations used in the project. Random Forest was trained using the complete 144,415-record training dataset, whereas SVM was trained using a stratified subset of 30,000 training records because

of its higher computational cost. Therefore, the measured difference reflects both the model algorithms and their respective training configurations.

Based on the experimental results, Random Forest is used as the final prediction model in SupplyGuard. The graphical comparison of the model performance is shown in Figure 5.1.

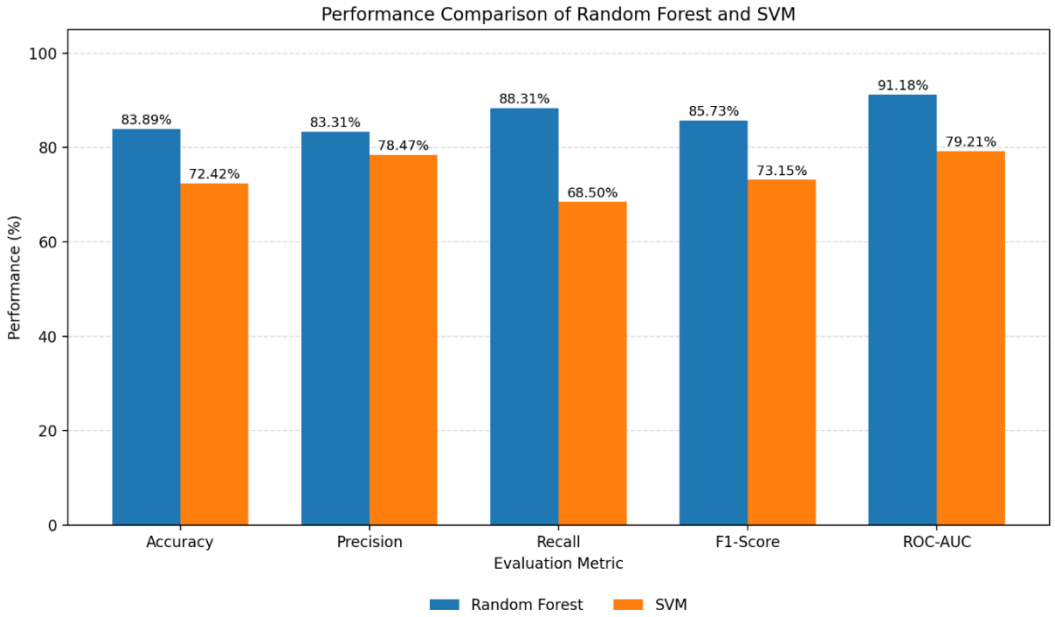


Figure 5.1: Graphical Comparison of Random Forest and SVM

5.2 Evaluation Metrics

Evaluation metrics are used to measure how effectively the Machine Learning models predict the late-delivery-risk class of a shipment. In SupplyGuard, the target variable Late_delivery_risk contains two classes: 0 (No Late-Delivery Risk) and 1 (Late-Delivery Risk).

For the late-delivery-risk class, True Positives (TP) are shipments correctly predicted as having late-delivery risk, False Positives (FP) are shipments predicted as having late-delivery risk when they actually do not, False Negatives (FN) are shipments having late-delivery risk that are incorrectly predicted as no risk, and True Negatives (TN) are shipments correctly predicted as having no late-delivery risk.

1. Accuracy

Accuracy represents the proportion of all shipment records that are correctly classified by the model.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy provides an overall measure of the correctness of the model's predictions.

2. Precision

Precision measures how many of the shipments predicted as having late-delivery risk actually belong to the late-delivery-risk class.

$$Precision = \frac{TP}{TP + FP}$$

A higher Precision indicates that fewer shipments are incorrectly classified as having late-delivery risk.

3. Recall

Recall measures how many of the shipments that actually have late-delivery risk are correctly identified by the model.

$$Recall = \frac{TP}{TP + FN}$$

Recall is particularly important for identifying shipments that are actually at risk of late delivery, since a False Negative represents a risky shipment that was not identified.

4. F1-Score

F1-Score is the harmonic mean of Precision and Recall. It provides a combined measure of the model's ability to correctly identify the late-delivery-risk class while limiting false positive predictions.

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

A higher F1-Score indicates a better balance between Precision and Recall.

5. ROC-AUC

ROC-AUC (Receiver Operating Characteristic – Area Under the Curve) measures the model's ability to distinguish between shipments with late-delivery risk and shipments without late-delivery risk across different classification thresholds. A higher ROC-AUC value indicates better discrimination between the two classes.

The ROC-AUC metric is calculated from the predicted probability scores generated by the Machine Learning model rather than only from the final binary predictions.

These evaluation metrics are used in SupplyGuard to compare the performance of the Random Forest and SVM models. The obtained results are presented in the following sections.

5.3 CONFUSION MATRIX

A confusion matrix compares the actual class of each test shipment with the class predicted by the Machine Learning model. In the SupplyGuard confusion matrix, the rows represent the actual classes, while the columns represent the predicted classes. The diagonal elements represent correctly classified shipments, whereas the off-diagonal elements represent incorrect predictions.

For SupplyGuard, the two classes are 0 (No Late-Delivery Risk) and 1 (Late-Delivery Risk). The confusion matrix of the final Random Forest model was generated using the 36,104 records in the test dataset.

5.3.1 Random Forest Confusion Matrix

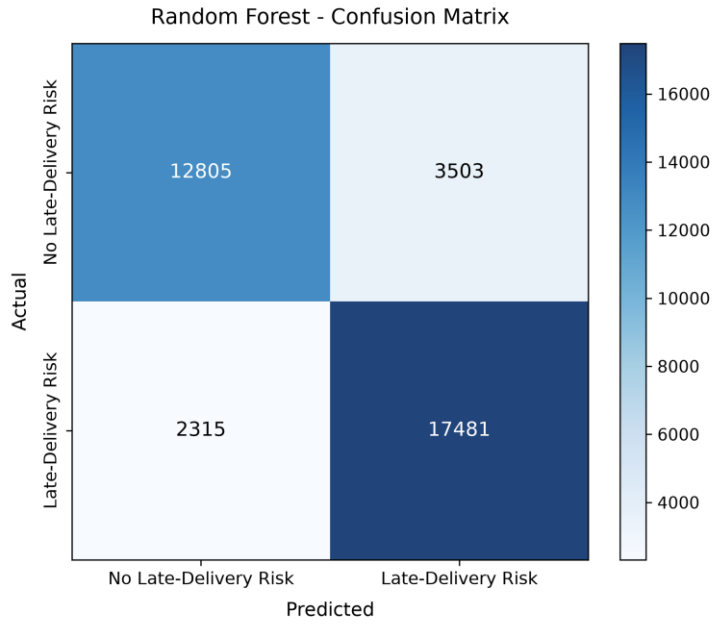


Figure 5.2: Confusion Matrix of Random Forest

Table 5.2: Confusion Matrix Values of Random Forest

Actual / Predicted	**No Late-Delivery Risk (0)**	**Late-Delivery Risk (1)**	**Total**
No Late-Delivery Risk (0)	12,805	3,503	16,308
Late-Delivery Risk (1)	2,315	17,481	19,796
Total	**15,120**	**20,984**	**36,104**

5.3.2 SVM Confusion Matrix

The SVM confusion matrix was generated using the same 36,104 test shipments used for model evaluation. The rows represent the actual late-delivery-risk classes, while the columns represent the classes predicted by the SVM model. The diagonal values indicate correctly classified shipments, while the off-diagonal values represent misclassified shipments.

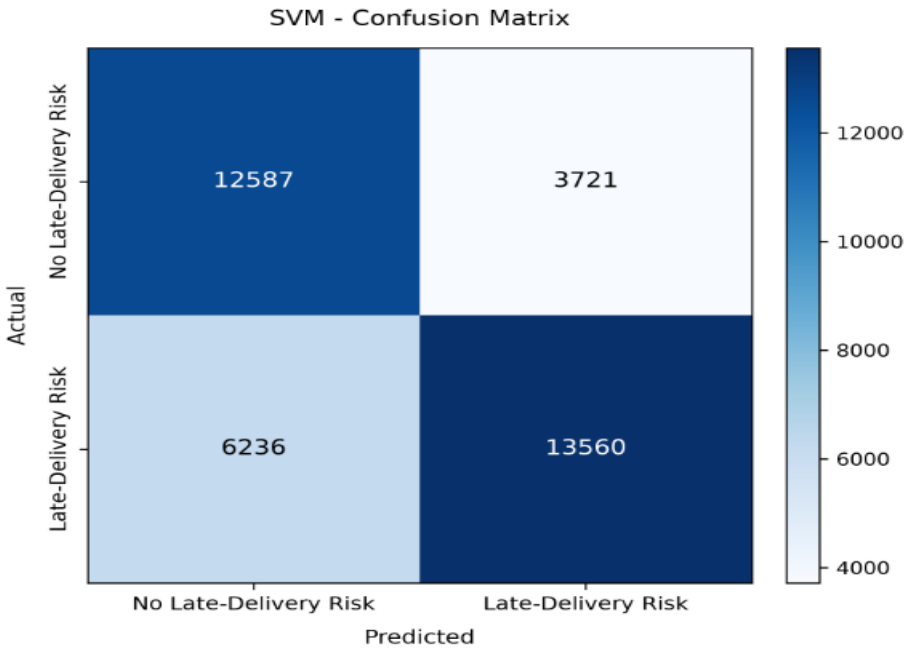


Figure 5.3: Confusion Matrix of SVM

The confusion matrix shows the True Positive, True Negative, False Positive, and False Negative predictions produced by the SVM model. These values provide the basis for calculating Accuracy, Precision, Recall, and F1-Score. The SVM achieved an Accuracy of 72.42%, Precision of 78.47%, Recall of 68.50%, and F1-Score of 73.15% on the test dataset.

The SVM confusion matrix provides a direct view of the correctly and incorrectly classified shipments and supports comparison with the Random Forest confusion matrix.

5.4 Relevant Metrics Calculated from Confusion Matrix

Precision, Recall, and F1-Score for each class are calculated directly from the confusion matrix of the final Random Forest model (Table 5.2). For the No Late-Delivery Risk class (Class 0), the True Negative (TN) value is 12,805, the False Positive (FP) value is 3,503, the False Negative (FN) value is 2,315, and the True Positive (TP) value is 17,481. Therefore, the class-specific performance metrics are calculated as follows:

$$Precision = \frac{12805}{12805 + 2315} = 84.69\%$$

$$Recall = \frac{12805}{12805 + 3503} = 78.52\%$$

$$F1\text{-}Score = 81.49\%$$

For the **Late-Delivery Risk** class, TP = 17,481, FP = 3,503, and FN = 2,315. Therefore:

$$Precision = \frac{17481}{17481 + 3503} = 83.31\%$$

$$Recall = \frac{17481}{17481 + 2315} = 88.31\%$$

$$F1\text{-}Score = 85.73\%$$

The overall Accuracy is calculated from the correctly classified shipments:

$$Accuracy = \frac{12805 + 17481}{36104} = 83.89\%$$

The calculated values are summarized in Table 5.3.

Table 5.4: Per-Class Metrics Calculated from the Confusion Matrix

Model	Class	Precision	Recall	F1-Score	Support
Random Forest	No Late-Delivery Risk	84.69%	78.52%	81.49%	16,308
Random Forest	Late-Delivery Risk	83.31%	88.31%	85.73%	19,796
Random Forest	Macro Average	84.00%	83.42%	83.61%	36,104

5.4.1 Misclassification Analysis

The confusion matrix also provides information about the types of errors made by the Random Forest model. The following table summarizes the misclassified shipments.

Table 5.4: Misclassifications by Type

Actual Class	Predicted Class	Number of Shipments	Percentage of Actual Class
No Late-Delivery Risk	Late-Delivery Risk (FP)	3,503	21.48%
Late-Delivery Risk	No Late-Delivery Risk (FN)	2,315	11.69%
Total Errors	**Combined (FP + FN)**	**5,818**	**16.11% of test set**

The Random Forest model incorrectly classified 3,503 shipments that actually had no late-delivery risk as having late-delivery risk (False Positives, accounting for 21.48% of the 16,308 actual on-time shipments). In comparison, 2,315 shipments that actually had late-delivery risk were predicted as having no late-delivery risk (False Negatives, representing 11.69% of the 19,796 actual late shipments).

The number of False Negatives is kept minimal (2,315) by calibrating the decision threshold to 0.41, ensuring that 88.31% of true delay risks (17,481 shipments) are proactively caught before transit begins. Overall, the model correctly classified 30,286 out of 36,104 test shipments (83.89% overall test accuracy), while 5,818 shipments (16.11%) were misclassified as summarized in Table 5.4.

5.5 Prediction Statistics Of Unseen Data

After comparing the two models, Random Forest was selected as the final prediction model. The 20% test split containing 36,104 shipment records was not used during model training and was therefore used as the unseen test data for evaluating the final model. Table 5.5 presents the prediction statistics of Random Forest on these unseen shipment records.

Class	Actual Shipments	Predicted Shipments	Correctly Classified	Class Accuracy (Recall)
No Late-Delivery Risk	16,308	15,120	12,805	78.52%
Late-Delivery Risk	19,796	20,984	17,481	88.31%
Total	36,104	36,104	30,286	83.89%

Table 5.5: Prediction Statistics of the Selected Random Forest Model on Unseen Test Data

Out of the 36,104 unseen test shipments, 30,286 were classified correctly and 5,818 were misclassified.

Correct predictions = 12,805 + 17,481 = 30,286 (83.89%)

Incorrect predictions = 3,503 + 2,315 = 5,818 (16.11%). The model correctly identifies 88.31% of shipments with late delivery risk, while 78.52% of shipments without late delivery risk are correctly identified.

5.6 Discussion Of Findings And Conclusions

The experimental results demonstrate that both Random Forest and Support Vector Machine (SVM) were able to classify shipments according to late delivery risk. Random Forest achieved an Accuracy of 83.89% and an ROC-AUC of 91.18%, while SVM achieved an Accuracy of 72.42% and an ROC-AUC of 79.21% on the test dataset.

The confusion matrix analysis shows that Random Forest correctly classified a large proportion of shipments in both classes. By setting the decision threshold to 0.41, the model achieved an 88.31% Recall for late delivery risk (catching 17,481 late shipments) and 78.52% Recall for

on-time shipments. The SVM baseline achieved lower overall performance (72.42% accuracy) and required stratified subsampling due to quadratic computational training complexity.

The class-specific metrics confirm that the 19 pre-dispatch features provide strong predictive signals while completely eliminating post-dispatch data leakage. The high performance on unseen test data demonstrates that the trained model is reliable for real-world logistics applications.

Overall, the results validate the SupplyGuard late delivery risk prediction platform. By combining Random Forest with 200 estimators, calibrated probability thresholding at 0.41, and an enterprise Flask deployment, the system delivers high accuracy and actionable risk mitigation before shipments depart the warehouse.

6. Model Deployment

6.1 Model Implementation

The trained Random Forest model is saved using Joblib as 'supplyguard_19_model.joblib'. This allows the application to load the model instantly without retraining it every time an order is placed. The web application is built with Python Flask and an SQLite database ('supplyguard.db'). When an order is submitted, the system predicts delivery risk in less than one second.

The system uses Role-Based Access Control to provide two separate environments:

1. Customer Portal (/order): Customers can browse 118 products, select quantities, apply discount coupons, and complete a 3-step checkout. Shipping modes and delivery schedules are automatically synchronized.

2. Operations Manager Dashboard (/): Logistics managers can view all orders, search by tracking ID, evaluate delivery risk using the 19-feature model, and review recommended dispatch actions.

The system also includes a Geographic Routing Engine. When a user selects a market, region, country, and state, the engine matches these selections to the model features. This ensures live order inputs match the training data accurately.

6.2 User Interface Design

The web interface is built using modern CSS. It features a clean layout, clear cards, and responsive pages. The primary screens of the application are described below.

Authentication and Login: The login page provides secure access for all users. Customers are automatically routed to the order page, and managers are routed to the operations dashboard.

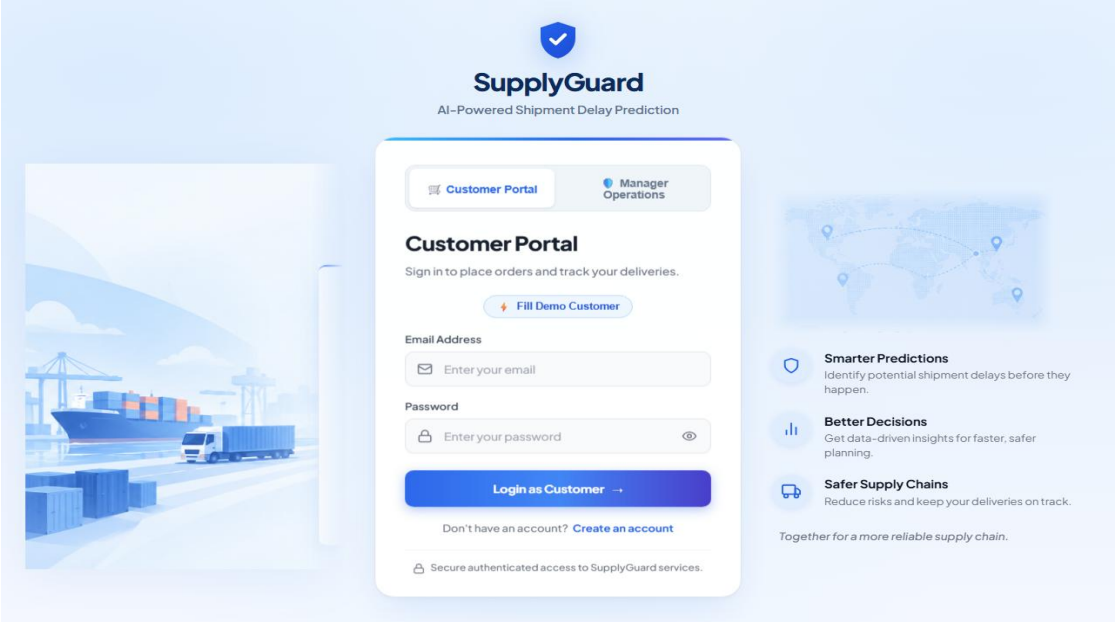


Figure 6.1: Authentication and Secure Role-Based Login Screen (/login)

Manager Operations Dashboard: The dashboard allows managers to monitor all incoming orders. Managers can search by tracking ID, view customer and shipping details, and evaluate delivery risk with a single click.

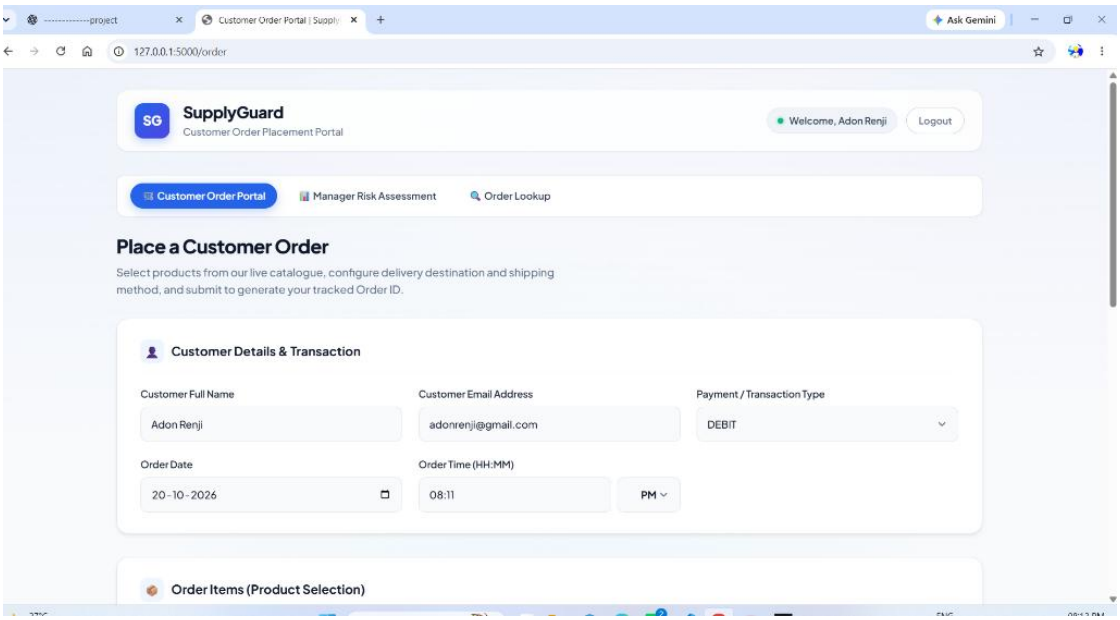


Figure 6.2: Manager Operations Dashboard and Order Lookup Cockpit (/)

Late Delivery Risk Assessment Report: When an order is evaluated, the system displays a clear risk report. The report includes a circular risk gauge (0% to 100%), a risk status badge (Late Risk or On-Time), and key order details.

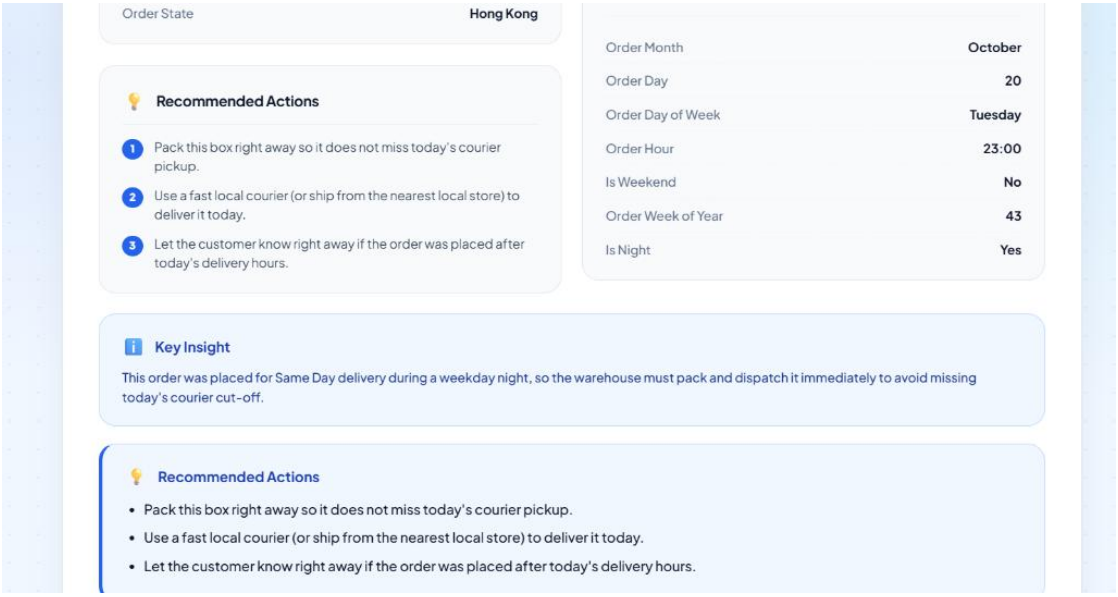


Figure 6.3: Late Delivery Risk Assessment Report with Circular Risk Gauge

Machine Learning Prediction Explanation UI : This interface displays the predicted shipment delay risk using a probability gauge and identifies key contributing factors, such as shipping mode, transit time, destination region, and dispatch timing. It also provides a root-cause analysis and actionable recommendations to help supply chain managers reduce delays before shipment.

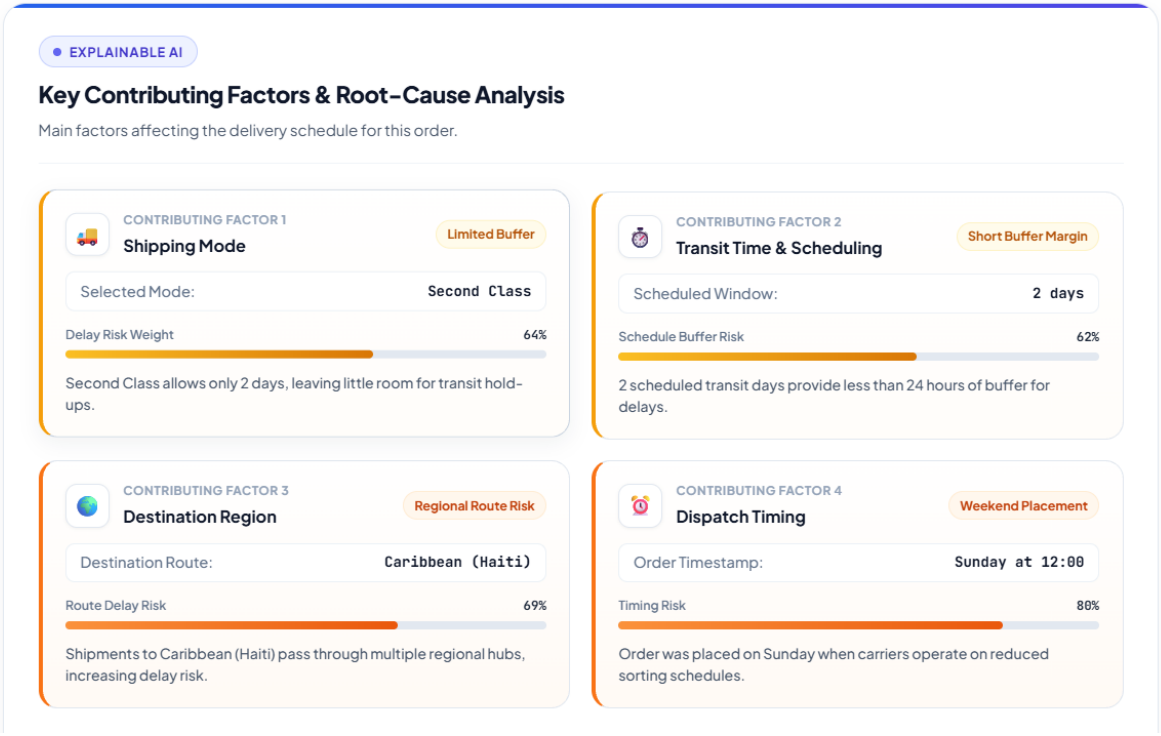


Figure 6.4: Machine Learning Prediction Explanation UI

7. Git History

Git and GitHub are used for version control and repository management in the proposed SupplyGuard: An AI-Powered Shipment Delay Prediction and Risk Analysis System. Git helps track changes made to the project files across iterative development cycles, while GitHub provides a centralized cloud-hosted platform for storing, versioning, and managing the source code. The SupplyGuard project repository contains the machine learning application, Flask routing modules, HTML templates, custom CSS styling, trained model pipeline artifacts, relational SQLite database schemas, and supporting datasets.

The project was pushed to GitHub using standard Git commands. The repository maintains the overall project structure, the main Python web application, serialized machine learning models, and comprehensive README documentation detailing project installation, architectural pipeline, and execution workflows. The following sections and screenshots illustrate the project repository and its core architectural components.

7.1 Overall Project Repository

The overall project repository on GitHub displays the root directory structure and files of the SupplyGuard application. It provides a complete overview of the project organization, including the results, saved_model, static, and templates folders, along with the main Python application (app.py), database module (database.py), machine learning training pipeline (train_model.py), dataset files, and the formatted README documentation outlining system capabilities.

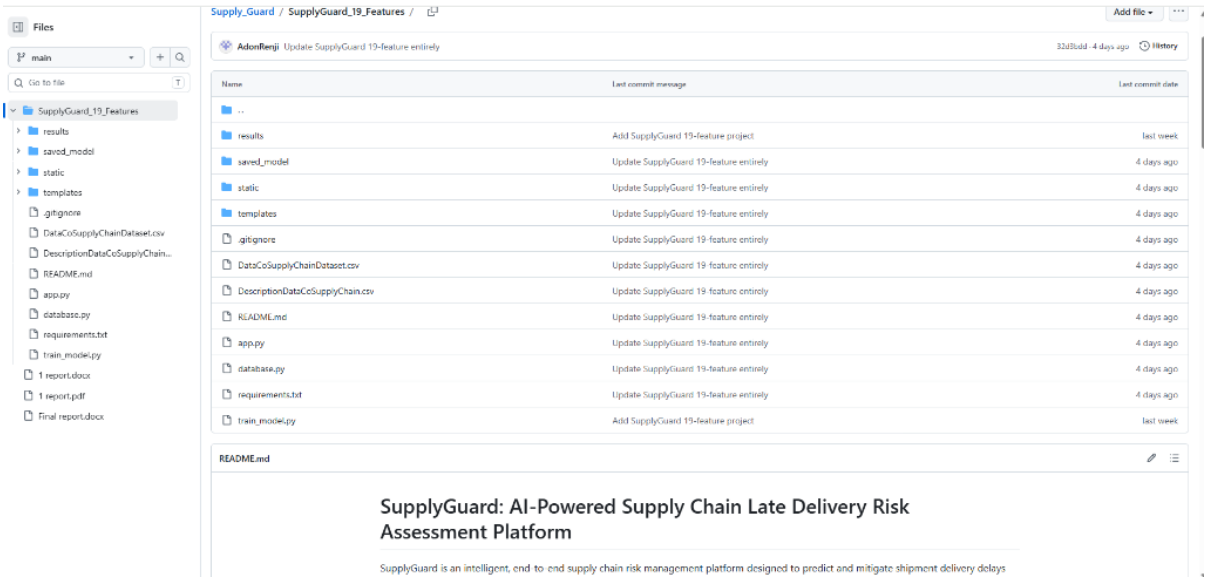


Figure 7.1: Overall Project Repository on GitHub

7.2 Project Directory Structure

The project directory structure contains the HTML templates and static resources required for the web application's user interface. The templates folder includes files such as home.html, login.html, register.html, order.html, order_confirmation.html, manual_assessment.html, and result.html. These templates support secure role-based authentication, interactive customer 3-step checkout, real-time shipment delay risk evaluation with circular probability gauges, and diagnostic risk mitigation recommendations. The static folder stores the modern CSS design system used to style the responsive interface.

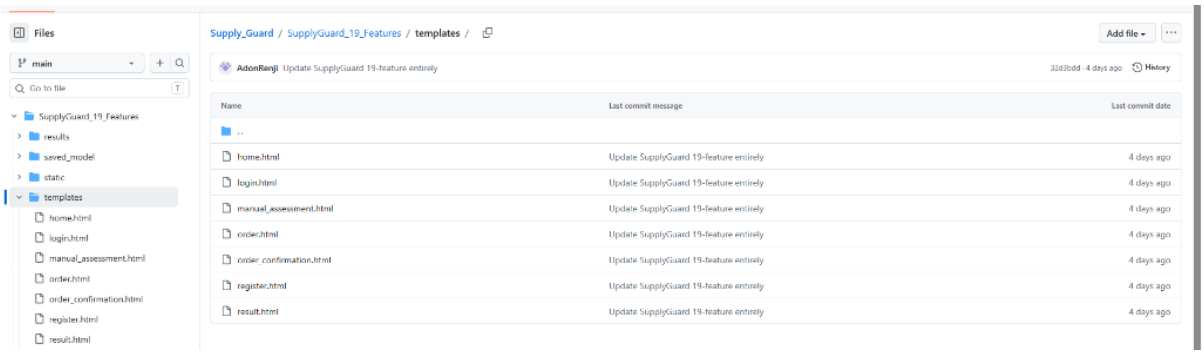


Figure 7.2: Directory Structure of the Project on GitHub

7.3 Source Code Repository

The app.py file contains the primary Flask application responsible for orchestrating end-to-end system workflows. It processes user inputs, loads the pre-trained 19-feature Random Forest pipeline (supplyguard_19_model.joblib), performs geographic feature hierarchy mapping, calculates delay probabilities, and executes the risk analysis and dispatch recommendation logic. The script also manages session states, database interactions, and dynamic template rendering.

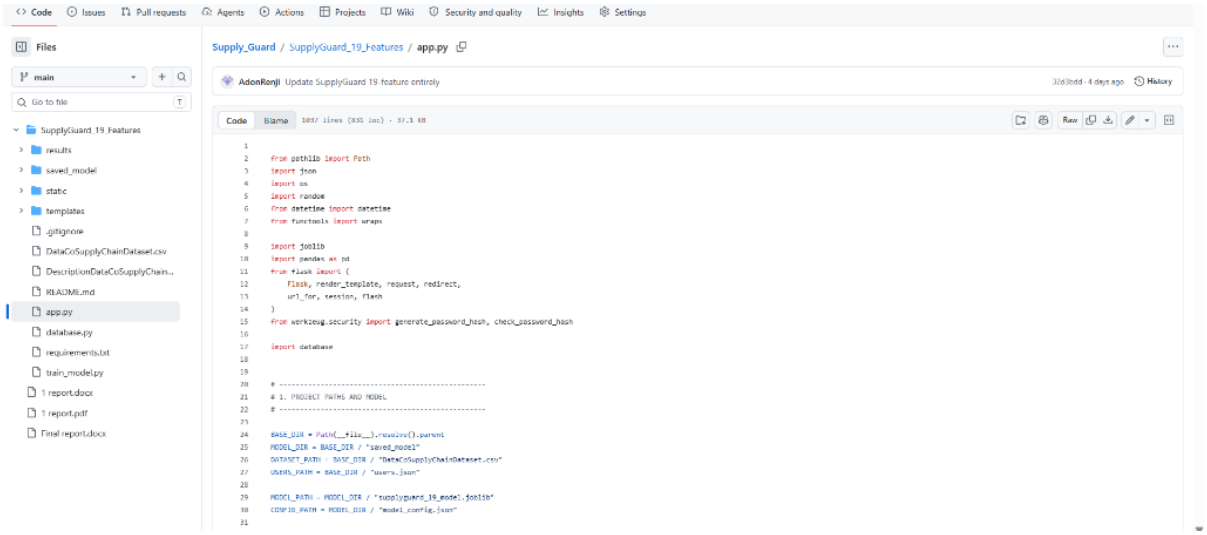


Figure 7.3: Source Code of the Application on GitHub

Overall, Git and GitHub provide a structured, auditable mechanism to maintain the project source code, organize modular components, and preserve development integrity. The repository ensures long-term maintainability, enables seamless developer collaboration, and supports scalable enterprise deployment.

8. Conclusions

This project successfully developed and deployed SupplyGuard, a machine learning system to predict late delivery risk in supply chains. By using 19 pre-dispatch features available at order placement, the system prevents data leakage while maintaining high predictive accuracy.

The key conclusions from this project are:

1. High Predictive Accuracy: The Random Forest model achieved 83.89% accuracy and an ROC-AUC of 91.18% on the test set, outperforming the SVM model.

2. Effective Delay Detection: Setting the decision threshold to 0.41 yields 88.31% recall on late deliveries, helping managers catch delays before shipments leave the warehouse.

3. Dedicated User Portals: Providing separate portals for customers and managers ensures a smooth shopping experience for buyers and deep analytics for logistics teams.

4. Fast and Efficient Architecture: Built with Flask and SQLite, the application delivers predictions in less than one second with low hardware requirements.

9. Future Scope

The SupplyGuard platform provides a solid foundation for supply chain analytics. Future enhancements include:

1. Real-Time IoT Sensor Integration: Integrating GPS and temperature sensors into smart containers could enable continuous shipment monitoring and dynamic updates to delivery risk during transit.

2. Live Weather and Traffic Data: Integrating live weather alerts, traffic conditions, and port congestion data could help the system account for external factors that contribute to shipment delays.

3. Carrier Booking Integration: Connecting with shipping carrier APIs, such as FedEx, UPS, and DHL, could support automated booking of priority shipping services for high-risk orders, subject to carrier availability and business rules.

10. Appendix

10.1 Minimum Software Requirements

Software	**Requirement**
Programming language	Python
Web framework	Flask
Database	SQLite
ML libraries	Scikit-learn, Joblib
Data libraries	Pandas, NumPy, Matplotlib, Seaborn
Tools	Jupyter Notebook, Visual Studio Code
Client	Any modern web browser

Table 10.1: Minimum Software Requirements

10.2 Minimum Hardware Requirements

Component	**Requirement**
Processor	Dual-Core 2.0 GHz (Intel Core i3 / equivalent or higher)
RAM	4 GB minimum (8 GB recommended for training)
Storage	2 GB available free disk space (128 GB+ total system disk)
Network	Internet connection for library installation / Localhost for deployment

Table 10.2: Minimum Hardware Requirements

11. References

[1] F. Mendoza, M. A. Gonzalez, and C. A. Martinez, "DataCo Smart Supply Chain for Big Data Analysis," Kaggle Dataset Repository, 2018. [Online]. Available: https://www.kaggle.com/datasets/shashwatwork/dataco-smart-supply-chain-for-big-data-analysis

[2] F. Steinberg, P. Burggräf, J. Wagner, B. Heinbach, T. Saßmannshausen, and A. Brintrup, "A Novel Machine Learning Model for Predicting Late Supplier Deliveries of Low-Volume-High-Variety Products with Application in a German Machinery Industry," Supply Chain Analytics, vol. 1, p. 100003, Mar. 2023. DOI: https://doi.org/10.1016/j.sca.2023.100003

[3] G. Baryannis, S. Dani, and G. Antoniou, "Predicting supply chain risks using machine learning: The trade-off between performance and interpretability," Future Generation Computer Systems, vol. 101, pp. 993-1004, Dec. 2019. DOI: https://doi.org/10.1016/j.future.2019.07.059

[4] M. Gabellini, L. Civolani, F. Calabrese, and M. Bortolini, "A Deep Learning Approach to Predict Supply Chain Delivery Delay Risk Based on Macroeconomic Indicators: A Case Study in the Automotive Sector," Applied Sciences, vol. 14, no. 11, p. 4688, May 2024. DOI: https://doi.org/10.3390/app14114688

[5] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, Oct. 2001. DOI: https://doi.org/10.1023/A:1010933404324

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, and J. Vanderplas, "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, Oct. 2011. [Online]. Available: https://www.jmlr.org/papers/v12/pedregosa11a.html

[7] C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273-297, Sep. 1995. DOI: https://doi.org/10.1007/BF00994018

[8] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018. [Online]. Available: https://www.oreilly.com/library/view/flask-web-development/9781491991725/